

Demir Bot: current specification and master roadmap

Current capabilities. Future work in implementation order.

Contents

Contents	2
Reading guide	7
Current package and evidence states	8
Architecture, routing and permission contracts	9
Coordination architecture: decision and evidence	10
Providers and implementation limits	11
Context, costs and completion	12
Retained assessment and installation evidence	13
Current skill catalogue	14
agent-configuration-review	14
architecture-review	14
clarify-and-execute	14
code-refactoring-refactor-clean	15
context-efficiency	15
deep-research-and-idea-validation	15
demir-bot	15
demir-linkedin	16
documentation-and-adrs	16
email-tone	17
frontend-quality-and-accessibility	17
git-and-github-workflow	17
jurisdiction-specific-legal-feasibility	17
linkedin-tone	18
ml-training-specialist	18
personal-writing	18
privacy-review	18
qa-and-test-evidence	19
release-readiness-and-observability	19
requirements-and-traceability	19
swiftui-performance-and-concurrency	20
youtube-learning	20

Completed guidance in the current plugin	21
Baseline measurement and honest cost attribution	21
Context overhead and observable loading	21
Routing telemetry	21
Output filtering discipline	21
Precise routing, negative cases and composition	22
Minimal skill catalogue and duplicate-rule reduction	22
Cache and freshness strategy	22
Source reuse, compact checkpoints and handoffs	22
Cost-benefit decision rule	23
User-set budgets, stop and graceful resume	23
Cost-aware model delegation	23
Proportional answers and visual output	23
Optional installation profiles	24
 Latest saved implementations and acceptance	 25
Product discovery and delivery	25
Repository onboarding	25
Error and type design	25
Broader GitHub operations	25
Content repurposing	25
Academic literature and paper review	25
User-directed learning	25
Learning controls and correction	26
Private lesson register	26
Controlled skill evolution	26
Structural repository checks	26
Behavioral regression and manual evaluation	26
Bounded improvement comparisons and effectiveness	27
On-demand lesson discovery	27
 New priority order and decision gates	 28
 A2 - Project-led engineering depth	 29
001. Swift concurrency, state ownership and identity	30
002. ios-project-structure	31
003. ios-architecture-patterns	32
004. dependency-injection-ios	33
005. swiftui-navigation	34
006. ios-error-handling	35
007. xcodebuild-error-taxonomy	36

008. derived-data-hygiene	37
009. spm-dependency-management	38
010. keychain-cryptokit	39
011. swiftdata-migrations	40
012. coredata-to-swiftdata	41
013. cloudkit-sync	42
014. ios-accessibility-audit	43
015. ios-localization	44
016. privacy-manifests	45
017. ios-security-hardening	46
018. Performance analysis and Apple profiling depth	47
019. xctest-patterns	48
020. swift-testing-patterns	49
021. ui-test-patterns	50
022. snapshot-testing	51
023. swiftlint-swiftformat-ios	52
024. simulator-device-management	53
025. Xcode CLI integration	54
026. Backend and API engineering	55
027. Database engineering	56
028. React/frontend depth	57
029. Stack-specific testing references	58
030. Project E2E and accessibility testing	59
031. UI design systems	60
032. Interaction and click-path reviews	61
033. Containers and reproducible environments	62
034. Operational observability	63
035. Model/API cost control	64
036. Typed connector development and App Store Connect	65
037. code-signing-provisioning	66
038. app-store-review-compliance	67
039. testflight-automation	68

A3 - Optional capabilities when a real need exists
69

040. Data and ML workflow depth	70
041. Cross-client memory	71
042. Local status dashboard	72
043. Notification integrations	73
044. Advanced testing modes	74
045. Additional language/framework packs	75
046. Mobile and on-device extensions	76
047. Specialist infrastructure	77
048. SEO and broader marketing	78

049. Networking and opportunity research	79
050. Investor/business documents	80
051. Extended social channels	81
052. Media and motion extras	82
053. Specialized research sources	83
054. Workspace and messaging connectors	84
055. widgetkit-live-activities	85
056. app-intents-shortcuts	86
057. background-tasks	87
058. push-notifications	88
059. universal-links-deep-linking	89
060. storekit2-subscriptions	90
061. coreml-on-device	91
062. foundation-models-ios	92
063. vision-speech-frameworks	93
064. metal-basics	94
065. avfoundation-media	95
066. Fastlane integration, optional	96
067. Xcode Cloud integration, optional	97
B - Integrate and refine	98
068. Action receipts and selective executable safeguards	99
069. Agent/runtime diagnostics	100
070. Upstream maintenance tooling	101
071. Bounded fresh-context review	102
072. Opt-in gate tiers and skip policy	103
073. Documentation maintenance	104
C - Comprehensive validation	105
074. Confidentiality release gate	106
D - Packaging and installation lifecycle	107
075. Dependency and support matrix	108
076. Supply-chain records	109
077. Licensing and redistribution decision	110
078. Safe private-profile onboarding	111
079. Installation doctor	112
080. Versioning and releases	113
081. Installation lifecycle tests	114
082. Scoped CI	115
083. Additional client support	116

E - Stable documentation and release decision	117
084. Installation guide	118
085. Update, uninstall and rollback guide	119
086. README and first-use quickstart	120
087. Everyday usage guide	121
088. Troubleshooting and support guide	122
089. Git/GitHub user recipes	123
090. Workflow cookbook	124
091. Examples and demos	125
092. Contributor and maintainer guides	126
093. Security reporting policy	127
094. GitHub collaboration setup	128
095. Multilingual documentation	129
096. Documentation site and richer presentation	130
097. Public-project community support	131
098. Release acceptance and recovery	132
F - Explicit deferrals and reuse decisions	133
099. Duplicate debugging specialist	134
100. Duplicate skill-authoring toolkit	135
101. Autonomous multi-agent loops	136
102. Background learning and persona mutation	137
103. Automatic maintenance and monitoring	138
104. Mandatory testing/verification loops	139
105. Cloud sync and university RAG	140
106. Training and compute infrastructure	141
107. Healthcare and regulated clinical packs	142
108. Crypto, trading and agent payments	143
109. Logistics, manufacturing and enterprise operations	144
110. Homelab and network administration	145
111. Duplicate provider implementations	146
112. ECC runtime and command parity	147
113. Early-window scheduling idea - reassess only	148
Repository artifact inventory and maintenance	149
Evidence gaps and next eligible work	153
Revision: 24 September 2026. Private instruction plugin. Capability-first planning; release polish last.	

Reading guide

This document describes the current Demir Bot plugin and the future roadmap. Completed guidance appears in the current-state section without roadmap numbers. Future work starts at Swift concurrency, state ownership and identity in A2. A1 general guidance is complete; optional and deferred work remains clearly marked.

Use the current item number and title together when starting a new chat. Copy its prompt and provide this document or the repository source at `docs/roadmap/demir-bot-master-roadmap.md`. Existing plugin references may contain historical item labels: identify the intended capability by its title and canonical source path. Those labels do not set the order of this roadmap. This document grants no permission to activate services, execute tests or publish changes.

Current package and evidence states

Plugin identity: demir-bot-pilot; display name: Demir Bot - Private Pilot; source version: 0.1.0+codex.20260924171748. The manifest is plugins/demir-bot-pilot/codex-plugin/plugin.json. The package exposes ./skills/ only. Source inspection found 22 SKILL.md files, 22 agents/openai.yaml metadata files, 137 total package files and 8 licence notices. There is one coordinating role (demir-bot), plus 21 specialists; personal-writing is a channel router within those specialists. YAML files describe invocation/display metadata, not running workers. The package now includes an opt-in Python structural validator and synthetic tests under qa-and-test-evidence/checks, plus behavioral cases and manual subject packets. It contains no executable publisher, memory service, custom subagent fleet, installer framework or autonomous daemon. These QA assets do not run on installation.

The development checkout and installed cache are distinct. The latest retained installation comparison found all 137 source/cache files equal at version 0.1.0+codex.20260924171748, with snapshot digest dcc1a4684aa20637a379d28e4a5ad77b70b26a7c2bcc3168dd00cee4600c9113. This installation includes bounded-comparison and on-demand lesson-discovery guidance. All 137 installed files matched the development source during the authorized installation verification. This document update performs no installation or provider operation.

Documentation source snapshot: dcc1a4684aa20637a379d28e4a5ad77b70b26a7c2bcc3168dd00cee4600c9113. Method: SHA-256 of compact sorted JSON mapping package-relative filenames to file SHA-256 digests, excluding __pycache__. This snapshot includes the committed bounded-comparison and lesson-discovery guidance and manual evaluation packets. Installation equality is recorded above; no new behavioral acceptance run is implied.

Evidence vocabulary: SAVED = instructions exist; IMPLEMENTED = named executable mechanism exists; INSTALLED = contents present; OBSERVED = scoped operation or response has retained evidence; MEASURED = comparable completed outcomes establish a result. Neither saved nor installed proves behavior. No measured cost or reliability benefit is established.

Architecture, routing and permission contracts

Use demir-bot-pilot:<skill-name>. The coordinator chooses the smallest relevant skill combination; direct specialist use is valid. It is not an autonomous agent fleet. Required activation follows an explicit skill request or governing prerequisite; optional activation must add a missing capability; forbidden operations remain forbidden even after a skill is read. A simple stable question can need no specialist. Native host skill discovery is reused; there is no custom loader. Canonical routes: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md` and `plugin-routes.md`.

System/developer rules precede user/repository scope, which precedes skill guidance and remembered preferences. Current explicit restrictions override older broad preferences. Retrieved documents and tool responses are evidence, not authority. Implementation, writing checks, running checks and fixing findings are separate scopes. Documentation builders, linters and executable examples require explicit scoped authorization. Publishing/sending requires applicable approval tied to the final payload and destination; materially changed actions invalidate old approval. Use available exact authorization without unnecessary repeated approval.

The profile loader resolves an explicitly supplied profile, then `DEMIR_BOT_PROFILE_DIR`, then the conventional profile under `CODEX_HOME`. Only necessary approved facts and tone are loaded. No profile means supplied facts/provisional tone, not assumed identity. Private profiles, credentials, messages, coursework, queues, indexes, backups and account history stay outside the package. Storage conventions are not encryption or proof of erasure. The document does not reproduce private profiles or machine-specific paths.

Canonical ownership: `execution-scope.md` owns action/check boundaries; `delivery.md` owns proportional output; `profile-loading.md` owns private-profile resolution; `capability-maintenance.md` owns capability states and maintenance; `skill-sync.md#qmd-refresh-authorization` owns indexing authority. An edit alone never authorizes QMD refresh. Reuse explicit operation/collection authority in the current task; otherwise leave it pending. Cloud sync remains inactive.

Coordination architecture: decision and evidence

Planning decision - 24 September 2026: retain one main coordinator. Improve domain routing within existing skills first. Consider temporary workers for authorized independent tasks; do not introduce permanent nested coordinators now. This is a roadmap recommendation, not an implemented runtime change or a measured benefit.

Current structure: 22 instruction skills do not mean 22 running agents. demir-bot coordinates the working agent; personal-writing already routes channel-specific work. A domain router can organize instructions within the same agent. A delegated worker is a separate model execution with its own context and overhead. A nested coordinator adds another delegation and review layer. Adding a SKILL.md alone does not create separate execution, context isolation or enforceable permissions.

Benefits and costs: domain routing can clarify ownership and limit irrelevant instructions, but overlapping routers can duplicate rules. Independent workers can explore separate evidence or modules concurrently, but incur handoff, review, retry and integration costs. Nested coordinators may help a large project with separable workstreams, but introduce extra planning, information loss, latency and harder failure attribution. Skill count alone is not a reason to add a management layer.

Research reviewed: Towards a Science of Scaling Agent Systems (version 3, April 2026) compares 260 configurations across six benchmarks. It reports that architecture must fit task structure: decomposable work can benefit while sequential planning can deteriorate, with overhead and diminishing returns in other settings. These benchmark findings do not predict Demir Bot gains. Source: <https://arxiv.org/abs/2512.08296v3>

Anthropic's multi-agent research report (June 2025) finds benefits for independent research branches, while noting shared-context dependencies and substantial token consumption. Its approximately 15-times token figure compares multi-agent use with ordinary chats, not the single-agent research baseline. This is vendor evidence from a specific system, not a Demir Bot cost estimate. Source: <https://www.anthropic.com/engineering/multi-agent-research-system>

AOchestra (February 2026 preprint) assembles workers using instructions, context, tools and model, supporting investigation of task-specific delegation rather than requiring a permanent management tree. Why Do Multi-Agent LLM Systems Fail? identifies specification, inter-agent alignment, verification and termination failures. These are research findings with benchmark/system limits, not proof of reliable nested coordination. Sources: <https://arxiv.org/abs/2602.03786v2> and <https://arxiv.org/abs/2503.13657>

Application to this roadmap: reuse completed A1 guidance and continue with current owners. Add a domain subcoordinator only after recurring work demonstrates a distinct coordination gap that an existing reference cannot cover. Start any authorized delegation experiment with one coordinator-to-worker layer. Prefer independent research questions or separate-module reviews; keep small edits, tightly coupled changes and unnecessarily shared private context with the parent. The parent owns scope, acceptance and final integration. Workers cannot expand permissions or approve external actions for one another.

Reuse the existing seven-field handoff, source-freshness, budget, measurement and delegation contracts in context-efficiency/references/source-reuse.md and runtime-options.md, under plugins/demir-bot-pilot/skills/. Define permitted inputs/actions, acceptance evidence and a stop condition. No recursive spawning, automatic evaluation, background teams or new provider installation follows from this decision.

Evaluation and revisit trigger: use the existing bounded independent-review guidance and Bounded improvement comparisons and effectiveness for a separately authorized comparison against the current single-coordinator baseline, reusing existing behavioral cases. Predeclare finite cases and limits; measure completed-task quality, privacy/permission failures, available total usage, latency, parent review, retries and coordination. Keep unknown costs explicit. Consider nested coordinators only if repeated evidence shows a parent bottleneck that simpler routing or one-layer delegation does not resolve. Retain the simpler design if additional layers do not justify their costs. Roadmap 112 remains deferred; no agents or model evaluations were launched for this document update.

Providers and implementation limits

External capabilities are optional and operation-specific: Firecrawl/current web research; official OpenAI documentation/developer tooling; Git/GitHub; Codex Security; Superpowers methods; mail/calendar; Google documents; Notion; Figma; Canva; media providers; Sites; Supabase and artifact tools. Their route names are hints, not installation, authentication or write-access claims. Resolve the actual current tool contract. If unavailable, use supplied evidence or an authorized local fallback, identify the missing operation and finish independent work. Do not install, authenticate, send private data or bypass a denial incidentally.

RTK is an optional output helper, not a global interception hook. QMD is optional local retrieval, not bundled indexing. Structural symbol/reference retrieval differs from prose search; do not claim compiler-resolved identity from keyword results. Buffer actions.md and publishing-runtime.md under skills/demir-linkedin/references/ describe approval/runtime boundaries but do not supply an executable connection. Apple, database, browser, training and hosting workflows require their actual project/tools/access. University RAG, cloud synchronization, model training and background automation remain deferred.

README.md and profile-loading.md contain historical fresh-discovery/validator statements. Later retained installation and assessment evidence is more specific; this document records it without silently updating those source files. No full supported-client matrix, redistribution decision, operational telemetry, comprehensive regression suite or complete lifecycle verification is established.

Context, costs and completion

context-efficiency owns bounded reads, selective freshness and source reuse. runtime-options.md owns honest measurement, output filtering/recovery, workflow choice, user budgets and delegation. source-reuse.md owns observable loading, opt-in telemetry, cache/source records and compact handoffs.

A baseline names task, result quality, model/settings, available usage/latency and missing attribution. Claims include failures, retries, worker calls, parent review and coordination, and require comparable completed outcomes. Output selection occurs before emission and preserves producer exit status, decisive errors, coverage and recoverable original evidence. Smaller text and RTK byte counts are not task-cost savings.

User budgets use available host mechanisms within their actual scope. Enforceable caps differ from estimates and shared-account limits; there are no arbitrary default budgets or continuous polling. Stop honestly with partial work and remaining requirements. Resume after relevant source/permission freshness checks. Delegation needs authorized independent work, supported models, bounded inputs/output and stop/escalation conditions; tightly coupled work stays together.

The single handoff format preserves goal, constraints, decisions, artifacts, evidence, unfinished work and next action. Persist only when authorized; no automatic progress file or memory service. Invalidate affected sources when content, versions, access, requirements or scope changes. Default answers are the shortest complete response with necessary evidence and unfinished-work disclosure. Visuals are used only when requested or materially helpful.

Retained assessment and installation evidence

Initial bounded assessment: snapshot d5caec83ee5414fd58bf47d320535472da8ead5a725f6016a703591bbf5aeb76; 31 grouped structural assertions passed, including 22 skill metadata records, 52 explicit relative links, 29 heading anchors and 44 namespaced route occurrences. This was not official host-schema certification. Twelve synthetic cases in two worker contexts met independent response expectations, once each, with zero case retries. One separate source reviewer read all skill bodies; references were selective. One artifact-write failure was retried with the available Python command; truncated reads required recovery.

Four fixes: documentation-and-adrs/SKILL.md now requires scoped execution authority; demir-linkedin and privacy-review use conditional visuals; skill-sync owns QMD refresh authorization with maintenance/freshness callers; capability-maintenance uses explicit sibling links to LinkedIn actions.md and publishing-runtime.md. Six files were changed.

Focused regression: snapshot d50044fe3584d95e77cbb85622dcfcf99cc66d7274332ac639b57286ea4e2fa2; source assertions improved from 1/9 to 9/9, with original eight failures retained; 17 affected links resolved. Six synthetic cases, one worker response each, met parent-reviewed expectations, zero case retries. Worker overhead: six orchestration calls and fourteen nested local commands; two truncations required three recovery reads. These are sampled responses under an independent safety harness, not real action enforcement, causal proof or repeated-sampling reliability.

Later installation verification records byte equality for all 106 files and source digest 2a24e24207dcb30712736ba9f3f99d98eb51d60564384d9e5cd52cdf8e5a9345 after the version-only update. Evidence retained and read for this revision: initial report.md, focused report.md and installation-verification JSON in temporary task storage. These temporary reports are not durable evidence archives; their private paths are deliberately omitted. Historical hashes and outcomes are transcribed here, not rerun.

Untested: actual publishing/QMD execution, live provider access, budget enforcement/cancellation, private-profile integration, full install/update/uninstall lifecycle, all domain workflows and fresh-task native routing. Exact aggregate token costs/model settings are unavailable. No benchmark, evaluation, plugin test or service operation ran during this document revision.

Current skill catalogue

Each entry describes saved instructions. Skill names below are exact; prefix each with demir-bot-pilot:. References and section headings describe scope, not proven execution. All paths are repository-relative. The manifest, metadata, skill bodies and reference inventory together describe the package; private dependencies are deliberately excluded.

agent-configuration-review

Responsibility and trigger: Review a requested agent, skill, hook or MCP configuration for permission, secret-handling, trust-boundary and supply-chain risks using scoped evidence. Use for explicit assistant-environment reviews; not automatic scans or application vulnerability audits.

Source: plugins/demir-bot-pilot/skills/agent-configuration-review/SKILL.md.

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: plugins/demir-bot-pilot/skills/agent-configuration-review/references/local-scanner.md;
plugins/demir-bot-pilot/skills/agent-configuration-review/references/provenance.md.

architecture-review

Responsibility and trigger: Review or design software architecture using requirements, repository evidence and explicit trade-offs. Use for system design, module boundaries, dependency cycles, data flow, architectural risks and significant structural decisions. Skip routine small fixes; a review does not authorize implementation.

Source: plugins/demir-bot-pilot/skills/architecture-review/SKILL.md.

Workflow coverage: Review workflow; Specialist boundaries; Proportional output.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: plugins/demir-bot-pilot/skills/architecture-review/references/review-checklist.md;
plugins/demir-bot-pilot/skills/architecture-review/references/sources.md.

clarify-and-execute

Responsibility and trigger: Turn rough or materially ambiguous requests into focused tasks through minimal clarification, preferably multiple-choice questions, then execute the original task. Use when Demir asks for help shaping a request, improving a prompt, choosing task scope, or when unresolved choices would materially change the result. Skip clarification for clear requests.

Source: plugins/demir-bot-pilot/skills/clarify-and-execute/SKILL.md.

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: plugins/demir-bot-pilot/skills/clarify-and-execute/references/choices.md;
plugins/demir-bot-pilot/skills/clarify-and-execute/references/sources.md.

code-refactoring-refactor-clean

Responsibility and trigger: Review and refactor existing code for maintainability, cognitive complexity, duplication, code smells, modularity, documentation, performance and test quality. Use for clean-code audits, safe restructuring and evidence-based quality reviews, including optional course traceability checks.

Source: `plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/SKILL.md`.

Workflow coverage: Workflow; Read only relevant detail; Targeted error and type review.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/contracts-and-documentation.md`;
`plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/course-lessons.md`;
`plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/quality-review.md`;
`plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/sources.md`;
`plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/targeted-review-provenance.md`;
`plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/testing-and-evidence.md`.

context-efficiency

Responsibility and trigger: Use for requested context/token efficiency, source reuse, handoffs or runtime optimization, and as Demir Bot's default method. Skip extra bookkeeping for simple questions and small edits.

Source: `plugins/demir-bot-pilot/skills/context-efficiency/SKILL.md`.

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md`;
`plugins/demir-bot-pilot/skills/context-efficiency/references/source-reuse.md`;
`plugins/demir-bot-pilot/skills/context-efficiency/references/sources.md`.

deep-research-and-idea-validation

Responsibility and trigger: Research complex questions and validate project ideas using primary sources, corroboration, counterevidence and decision-focused analysis. Use for substantial research, technology comparisons, feasibility, demand and competitive analysis; skip simple factual answers and routine coding.

Source: `plugins/demir-bot-pilot/skills/deep-research-and-idea-validation/SKILL.md`.

Workflow coverage: Scope and intake; Research with evidence; Validate a project idea; Deliver and preserve boundaries.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/deep-research-and-idea-validation/references/provenance.md`.

demir-bot

Responsibility and trigger: Use for explicit Demir Bot requests, cross-workflow coordination, or code creation/changes. Route to relevant skills; skip unrelated standalone questions.

Source: `plugins/demir-bot-pilot/skills/demir-bot/SKILL.md`.

Workflow coverage: Active user profile; Execute with focused context; Binding execution scope; Clickable preference choices; Available plugins and built-in capabilities; Capability tracking and skill maintenance; Git and GitHub work; Preserve evidence, reduce waste; Personalization and actions; Deliver clearly.

Relationships: Coordinates only relevant specialists; context-efficiency is its default method.

References: plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md;
plugins/demir-bot-pilot/skills/demir-bot/references/capability-register.md;
plugins/demir-bot-pilot/skills/demir-bot/references/delivery.md;
plugins/demir-bot-pilot/skills/demir-bot/references/execution-scope.md;
plugins/demir-bot-pilot/skills/demir-bot/references/personal-workflows.md;
plugins/demir-bot-pilot/skills/demir-bot/references/plugin-routes.md;
plugins/demir-bot-pilot/skills/demir-bot/references/profile-loading.md;
plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;
plugins/demir-bot-pilot/skills/demir-bot/references/skill-sync.md;
plugins/demir-bot-pilot/skills/demir-bot/references/upstream-adaptations.md.

demir-linkedin

Responsibility and trigger: Develop the user's LinkedIn presence through current CS, data science, AI and LLM research, evidence-based posts, profile positioning, relevant networking, comments and analytics. Use for LinkedIn drafts, news-to-post requests, employer visibility or audience growth. Publish or contact people only after approval of the specific final action and with supported access.

Source: plugins/demir-bot-pilot/skills/demir-linkedin/SKILL.md.

Workflow coverage: Working contract; Apply the approved voice; Select the relevant work; Approval and publishing; Delivery and reuse.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: plugins/demir-bot-pilot/skills/demir-linkedin/references/actions.md;
plugins/demir-bot-pilot/skills/demir-linkedin/references/approval-ledger.md;
plugins/demir-bot-pilot/skills/demir-linkedin/references/growth.md;
plugins/demir-bot-pilot/skills/demir-linkedin/references/publishing-runtime.md;
plugins/demir-bot-pilot/skills/demir-linkedin/references/research-and-content.md;
plugins/demir-bot-pilot/skills/demir-linkedin/references/sources.md.

documentation-and-adrs

Responsibility and trigger: Maintain project documentation and architectural decision records as Demir's Documentation Officer. Use for requested documentation creation/review, significant design decisions, changed public APIs or setup instructions, and inconsistencies across requirements, design, implementation and testing evidence. Keep small unrelated coding tasks lightweight.

Source: plugins/demir-bot-pilot/skills/documentation-and-adrs/SKILL.md.

Workflow coverage: Work from evidence; Load only needed references; Keep maintenance proportionate.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: plugins/demir-bot-pilot/skills/documentation-and-adrs/references/adr-and-templates.md;
plugins/demir-bot-pilot/skills/documentation-and-adrs/references/consistency-and-evidence.md;
plugins/demir-bot-pilot/skills/documentation-and-adrs/references/sources.md.

email-tone

Responsibility and trigger: Apply the user's approved email tone to requested drafts and revisions, keeping channel preferences separate from shared facts and delivery authorization.

Source: `plugins/demir-bot-pilot/skills/email-tone/SKILL.md`.

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: No local reference directory; consult the skill body and its canonical shared/private pointers..

frontend-quality-and-accessibility

Responsibility and trigger: Implement usable, accessible web UI in the actual project stack, covering semantics, keyboard and focus behavior, forms, responsive layouts, interface states and reduced motion. Use for relevant frontend implementation or explicitly requested UI/accessibility reviews; do not trigger automatic audits or tests.

Source: `plugins/demir-bot-pilot/skills/frontend-quality-and-accessibility/SKILL.md`.

Workflow coverage: Apply within the requested change; Semantics and perceivable content; Keyboard and focus; Forms and feedback; Responsive layout and interface states; Reuse available providers appropriately; Verification and completion boundaries.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/frontend-quality-and-accessibility/references/provenance.md`.

git-and-github-workflow

Responsibility and trigger: Inspect and perform scoped Git and GitHub repository work, including branches, focused commits, synchronization, pull requests and recovery from conflicts or failed operations. Use for requested version-control work or a concrete Git blocker; ordinary code edits do not imply committing, pushing or merging.

Source: `plugins/demir-bot-pilot/skills/git-and-github-workflow/SKILL.md`.

Workflow coverage: Scope and authorization; Inspect before changing state; Preserve work and choose a branch; Focused staging and commits; Fetch and integration strategy; Pushes, pull requests and merges; Finish with evidence.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/git-and-github-workflow/references/provenance.md`;
`plugins/demir-bot-pilot/skills/git-and-github-workflow/references/recovery.md`.

jurisdiction-specific-legal-feasibility

Responsibility and trigger: Research legal feasibility for a defined activity, jurisdiction and relevant date using current official sources. Use for product/business legal questions, regulatory applicability and questions for counsel; not certification, representation or external legal action.

Source: `plugins/demir-bot-pilot/skills/jurisdiction-specific-legal-feasibility/SKILL.md`.

Workflow coverage: Define the actual case; Reuse research and privacy workflows; Establish authority and applicability; Analyze feasibility; Output and action boundaries.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: [plugins/demir-bot-pilot/skills/jurisdiction-specific-legal-feasibility/references/provenance.md](#).

linkedin-tone

Responsibility and trigger: Apply the user's approved linkedin tone to requested drafts and revisions, keeping channel preferences separate from shared facts and delivery authorization.

Source: [plugins/demir-bot-pilot/skills/linkedin-tone/SKILL.md](#).

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: No local reference directory; consult the skill body and its canonical shared/private pointers..

ml-training-specialist

Responsibility and trigger: Implement or review ML data pipelines, training and fine-tuning workflows in the actual project stack, or assess whether training would improve Demir Bot. Use for concrete ML work and assistant-improvement research; do not start training or autonomous self-modification merely by invoking the skill.

Source: [plugins/demir-bot-pilot/skills/ml-training-specialist/SKILL.md](#).

Workflow coverage: Choose an intervention; Implement in the actual stack; Execution and delivery.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: [plugins/demir-bot-pilot/skills/ml-training-specialist/references/provenance.md](#).

personal-writing

Responsibility and trigger: Write or revise the user's emails, applications, messages and personal prose using shared approved facts and voice examples. Use for writing in the user's voice and maintaining explicitly approved reusable facts or samples; route LinkedIn-specific research and actions to demir-linkedin.

Source: [plugins/demir-bot-pilot/skills/personal-writing/SKILL.md](#).

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Routes to email-tone or linkedin-tone; LinkedIn operations use demir-linkedin.

References: [plugins/demir-bot-pilot/skills/personal-writing/references/facts-and-voice.md](#).

privacy-review

Responsibility and trigger: Review personal-data handling in a requested feature, assistant workflow, integration or data flow. Use for privacy reviews and concrete decisions about collection, tool sharing, telemetry, retention, deletion, consent or reuse of personal information. Route vulnerability audits to existing Codex Security skills. Skip unrelated work and automatic audits after routine coding tasks.

Source: [plugins/demir-bot-pilot/skills/privacy-review/SKILL.md](#).

Workflow coverage: Choose the right workflow; Inspect only the relevant data flow; Boundaries.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: [plugins/demir-bot-pilot/skills/privacy-review/references/sources-and-coverage.md](#).

qa-and-test-evidence

Responsibility and trigger: Plan and implement risk-based tests, review test quality, investigate coverage gaps and produce honest execution evidence. Use only for explicitly requested QA strategy, test creation/execution, integration/export/persistence verification or evidence audits. Skip unnecessary test infrastructure for trivial low-impact edits.

Source: [plugins/demir-bot-pilot/skills/qa-and-test-evidence/SKILL.md](#).

Workflow coverage: Focused workflow and boundaries in the source body.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: [plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/demir-bot-evaluation.md](#);
[plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/evidence.md](#);
[plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/sources.md](#);
[plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/test-design.md](#).

release-readiness-and-observability

Responsibility and trigger: Assess release readiness or implement requested deployment and observability changes using project-specific evidence, configuration, migration, rollback, health, logging, alerting and recovery guidance. Use for explicit release preparation or operational tasks; no automatic post-merge audit, tests or deployment.

Source: [plugins/demir-bot-pilot/skills/release-readiness-and-observability/SKILL.md](#).

Workflow coverage: Establish scope and authorization; Readiness evidence; Deployment prerequisites and configuration; Migrations and data integrity; Health signals and observability; Rollback and recovery; Completion.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: [plugins/demir-bot-pilot/skills/release-readiness-and-observability/references/provenance.md](#).

requirements-and-traceability

Responsibility and trigger: Turn project ideas, assignment criteria and change requests into measurable requirements and acceptance criteria; audit ambiguity, scope, bidirectional traceability and change impact. Use for substantial unclear features, requirements documents, requirement-to-test mappings or conflicting specifications. Skip routine self-contained fixes and unrelated coding tasks.

Source: [plugins/demir-bot-pilot/skills/requirements-and-traceability/SKILL.md](#).

Workflow coverage: Establish scope and evidence; Specify; Handoff and maintenance.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: [plugins/demir-bot-pilot/skills/requirements-and-traceability/references/requirements.md](#);
[plugins/demir-bot-pilot/skills/requirements-and-traceability/references/sources.md](#);
[plugins/demir-bot-pilot/skills/requirements-and-traceability/references/traceability.md](#).

swiftui-performance-and-concurrency

Responsibility and trigger: Implement or review SwiftUI state, view-update performance and safe Swift concurrency for Apple-platform projects. Use for relevant SwiftUI changes, task lifecycle, isolation diagnostics or performance problems; follow actual toolchain settings and do not automatically build, profile or test.

Source: `plugins/demir-bot-pilot/skills/swiftui-performance-and-concurrency/SKILL.md`.

Workflow coverage: Establish project context; State ownership and view updates; Expensive work and evidence; Task lifetime and cancellation; Actor isolation and safe sharing; Delivery and boundaries.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/swiftui-performance-and-concurrency/references/provenance.md`.

youtube-learning

Responsibility and trigger: Understand and apply lessons from requested YouTube videos using timestamped transcript and visual evidence. Use for video explanations, tutorials, lectures, comparisons and learning from demonstrations; distinguish transcript-only access from actual visual analysis. Not an autonomous watcher or model-training workflow.

Source: `plugins/demir-bot-pilot/skills/youtube-learning/SKILL.md`.

Workflow coverage: Scope and evidence; Understand and apply; Learning and privacy boundaries.

Relationships: Selected directly or through demir-bot; inherits current task authorization. Combine with another specialist only for a distinct missing requirement.

References: `plugins/demir-bot-pilot/skills/youtube-learning/references/provenance.md`;
`plugins/demir-bot-pilot/skills/youtube-learning/references/providers.md`.

Completed guidance in the current plugin

Status for all thirteen: SAVED / INSTALLED at the recorded snapshot; scoped evidence only; broader behavior and benefit UNVERIFIED. These are not new services or thirteen added skills. Reuse them during future work.

Baseline measurement and honest cost attribution

Saved guidance: Before optimizing, define representative task outcomes and record already available usage, elapsed time, retries and failures. Separate raw tokens, cached tokens, API cost estimates and subscription allowance. Mark unavailable values and concurrent-account activity. Design any additional comparison now; run it only under a separate explicit evaluation request.

Acceptance contract: A baseline record names the task, result quality, model/settings, available metrics and missing attribution. Any savings claim includes a comparable completed-task result and all retries/coordination. Until an authorized comparison exists, label the improvement unverified.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Context overhead and observable loading

Saved guidance: Audit repeated instructions, unnecessary skill activation, oversized tool output and observable context sources. Make the coordinator a minimal router; preserve bindings and move conditional details into focused references. Measure only under separately authorized comparison scope.

Acceptance contract: Identify observed loaded instructions, metadata, tool descriptions and returned output where visible; label estimates and unknowns. Name a specific removable duplication without losing requirements. A smaller document alone does not prove a cheaper completed task.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/source-reuse.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Routing telemetry

Saved guidance: Design minimal opt-in records of expected versus actual skill activation, with privacy-safe retention; do not install a conversation observer.

Acceptance contract: For scoped observed examples, distinguish expected from actual skill activation, unnecessary loads and unavailable trace data. Records exclude private content by default; no background observer is installed. Written telemetry guidance alone is not operational telemetry.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/source-reuse.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Output filtering discipline

Saved guidance: Return selected fields and relevant passages before tool output enters context; preserve errors, coverage and recoverable full evidence. Apply existing RTK selectively, not as a universal wrapper.

Acceptance contract: The selected output path preserves exit status, decisive errors and a recoverable original. Show the intended bounded output and recovery mechanism; actual session savings remain unverified without an authorized comparison. Reuse installed RTK where suitable.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Precise routing, negative cases and composition

Saved guidance: Verify that Demir Bot selects relevant installed skills, including Superpowers, without redundant tagging or loading entire frameworks. Reconcile task scope, testing, approval and delegation requirements using instruction precedence. Check missing-provider fallback and installation versus access versus operation evidence. Reuse existing providers before creating new specialists. Reuse existing routing, knowledge, tool, permission and model failure categories; define actionable distinctions rather than creating another diagnostic framework. Add synthetic cases for when a skill must not activate, including simple edits, already-known context and unavailable providers. Define the smallest adequate combination of existing specialists; avoid duplicated context and incompatible procedural requirements.

Acceptance contract: Define required, optional and forbidden activation; resolve instruction conflicts, missing-provider fallback and no-specialist cases without unnecessary framework loading.

Saved owner: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Minimal skill catalogue and duplicate-rule reduction

Saved guidance: Codex already loads skill bodies on demand. Improve concise names/descriptions, narrow activation and canonical ownership of repeated rules. Inspect overhead before adding a new loading mechanism.

Acceptance contract: Each revised trigger has clear positive and negative examples and one canonical owner for shared rules. Native lazy loading is reused. Report saved guidance separately from observed activation and measured context reduction.

Saved owner: plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Cache and freshness strategy

Saved guidance: Reuse relevant unchanged source evidence, documentation and scoped indexes; define invalidation and bounded storage without promising control of hidden model caches. Preserve useful DerivedData, dependency caches and scoped QMD state; invalidate on relevant changes rather than routine cleanup. No unsolicited build or index expansion. Read changed areas and relevant dependencies, review scoped diffs and update only affected indexes. Run changed-target tests/builds only when requested. For larger repositories, assess an on-demand, token-bounded symbol/reference or dependency map using existing tools first. Account for indexing and map-loading overhead; tiny tasks should use direct search. Validate the actual Swift/project compatibility before recommending any external integration.

Acceptance contract: A selected cache or retrieval mechanism records its source/version, invalidation conditions and original-evidence path. Structural retrieval distinguishes symbols/references from prose search. Guidance, connected tooling and measured benefit are reported separately; no blanket indexing or provider installation.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/source-reuse.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Source reuse, compact checkpoints and handoffs

Saved guidance: Strengthen existing context-efficiency, QMD and RTK use with narrow retrieval, freshness checks and concise resumable handoffs. Preserve unresolved requirements and source evidence. Link to Context overhead and observable loading; do not create redundant context ledgers or an always-on memory service. Preserve decisions, constraints, unfinished work and source locators while removing redundant narrative; use summaries without replacing original evidence when precision matters. Reuse the existing compact handoff: goal, constraints, decisions, artifacts, evidence, unfinished work and next action; persist only when authorized.

Acceptance contract: Use one minimal handoff format and precise freshness rules; do not introduce automatic progress files.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/source-reuse.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Cost-benefit decision rule

Saved guidance: Choose the smallest adequate workflow using expected task quality, total usage, latency and risk; avoid speculative numerical savings or mandatory preflight ceremonies.

Acceptance contract: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

User-set budgets, stop and graceful resume

Saved guidance: Respect an explicit user task budget using available host controls and sparse usage snapshots. Preserve quality; stop with an honest checkpoint if the budget cannot cover the task. Explain shared-account attribution and soft versus enforceable caps.

Acceptance contract: Reuse available host budget mechanisms; implement only enforceable scope and report partial work. No arbitrary daily caps, continuous polling or unnecessary estimate approval.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Cost-aware model delegation

Saved guidance: For explicitly authorized delegation, select a supported cheaper model for a bounded independent task and include parent review, retries and handoff cost. Compare against single-agent work before recommending a default. Escalate task complexity to an appropriate available model or the user only when needed; distinguish missing facts, permissions and model capability.

Acceptance contract: The delegation policy names eligible independent tasks, excluded tightly coupled tasks, supported model choices, a bounded handoff and an escalation/stop rule. Any comparison counts parent review, worker calls, failures and retries at comparable quality. No lower-cost claim is made from model price or agent count alone.

Saved owner: plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Proportional answers and visual output

Saved guidance: Default to the shortest complete response. Use visual output only when it materially improves understanding or is explicitly requested; revise the current automatic visual-inventory preference within the specific approved change. Preserve necessary evidence and unfinished-work disclosure.

Acceptance contract: Simple answers need no generated visual or long completion checklist; complex comparisons retain useful detail.

Saved owner: plugins/demir-bot-pilot/skills/demir-bot/references/delivery.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Optional installation profiles

Saved guidance: Core, coding, academic and writing selections if bundle growth justifies them. Preserve stable names and required references; no premature installer framework.

Acceptance contract: Stable names and reference closure survive selection; complexity is justified over a simple full instruction bundle.

Saved owner: plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Later changes require scoped authorization; runtime criteria are not certified by saved text.

Latest saved implementations and acceptance

The following capabilities are complete as source deliverables, not claims of verified runtime behavior. The skill count remains 22; no coordinator or specialist was added. Private preferences, personal lessons and accounts were not modified.

Product discovery and delivery

SAVED: requirements-and-traceability owns user jobs, capability maps, MVP scope, decision/implementation/acceptance ownership and observable acceptance. Architecture-review supplies technical feasibility and boundary decisions. Unknown owners, demand and dependencies stay explicit. Sources: skills/requirements-and-traceability/references/requirements.md and skills/architecture-review/SKILL.md under plugins/demir-bot-pilot. No new general coordinator.

Repository onboarding

SAVED: architecture-review maps entry points, conventions and technical/human ownership, and traces a source-supported request/data flow. Source tracing is distinguished from observed runtime execution; persistent onboarding documents require a request. Source: plugins/demir-bot-pilot/skills/architecture-review/references/review-checklist.md.

Error and type design

SAVED: code-refactoring-refactor-clean covers silent failure, invalid states, nullability, boundary contracts and recovery. Existing ECC reviewer perspectives inform the same-agent review; no mandatory workers. Sources: its references/contracts-and-documentation.md and targeted-review-provenance.md. No application tests or runtime claims.

Broader GitHub operations

SAVED: existing git-and-github-workflow owns issue triage, CI diagnosis, release preparation and security alert triage through references/github-operations.md. Read-only investigation is separate from comments, reruns, releases, merges and remediation. Provider access and incomplete log coverage remain explicit; no live GitHub operations were performed for this implementation.

Content repurposing

SAVED: personal-writing owns approved-source adaptation into channel drafts, series and portfolios; demir-linkedin links to the shared method. references/content-repurposing.md preserves facts, team/academic credit, confidentiality, voice and reuse permissions. Repurposing approval does not authorize publishing.

Academic literature and paper review

SAVED: deep-research-and-idea-validation/references/academic-review.md defines proportional protocols, search logs, deduplication, exclusion reasons, evidence tables, methods/baselines and citation support. Preprints, reviews, primary studies and unavailable full text stay distinct. No separate research engine or automatic university RAG.

User-directed learning

SAVED: demir-bot/references/capability-maintenance.md separates observed mistakes from uncertain causes and response-only feedback from authorized persistence. It identifies the correct preference, project or skill owner; preserves minimal provenance and genuine scoped recovery. No private preferences were changed.

Learning controls and correction

SAVED: the same maintenance reference defines inspect, revise, forget and rollback controls. Current authority and scope govern contradictions; retrieved recency is not approval. Deletion claims distinguish current records from historical or inaccessible copies and must not recreate withdrawn data as a default backup.

Private lesson register

SAVED FORMAT ONLY: minimal lesson fields cover owner, trigger, action, source, scope, approval and supersession. Project lessons stay project-scoped; confidence and silence never grant approval. No register was created or populated and no memory service was installed.

Controlled skill evolution

SAVED: approved lessons can support an on-request, focused existing-skill change with reviewable wording/diff, preserved adaptations, provenance and rollback. Source edits, installation and indexing have separate permissions. No private-history mining or evolution cycle occurred.

Structural repository checks

IMPLEMENTED AND ACCEPTED WITHIN SCOPE: qa-and-test-evidence/checks includes a local manifest schema, independently maintained inventory, metadata/link/route checks, pinned tooling and synthetic regression cases. Exact CPython 3.11.11 and pinned dependencies were provisioned outside Git. The first run passed 21 synthetic cases but failed the real-package case on 11 optional-metadata findings. One correction batch aligned optional fields with inspected authoring guidance and added two regressions; supplied invalid fields and incorrect routes remain rejected.

The final run passed 24/24 cases (23 synthetic plus current-source acceptance); its source report had zero findings across 118 files and 22 skills. Both final commands exited 0; original failures and exit 1 outputs were retained. Exactly one correction batch and one rerun of each command occurred. Assessed snapshot:

0c1de15b537cdd06775907bb4566fdf18dd9cc49391a5c3fd02c23e830b78e04. Later manual evaluation assets are outside that historical acceptance. This is a local structural contract, not official host-schema certification, privacy assurance or full runtime reliability.

Behavioral regression and manual evaluation

COMPLETED UNDER USER-ACCEPTED TEN-CASE SCOPE: The user completed ten distinct manual cases with Demir Bot explicitly tagged and accepted their results. Coverage included concise arithmetic, a minimal patch, evidence-limited comparison, publication approval boundaries, confidential-data omission, hostile document instructions, fictional email drafting, CI uncertainty, honest cost/test claims and response-only preferences. The accepted campaign supersedes the earlier larger manual plan; no additional cases or repetitions are required for that accepted scope.

The existing behavioral cases, reviewer protocol and manual packets remain reusable QA assets. Screenshots support the visible responses and listed skill activity. This acceptance is scoped to the ten cases and does not establish universal reliability or measured improvement. No new model evaluations were run for this update.

Bounded improvement comparisons and effectiveness

SAVED: qa-and-test-evidence/references/demir-bot-evaluation.md now defines a bounded before/after comparison using a concrete instruction change, independent success criteria, comparable source versions and conditions, observable outcomes, available task-attributed usage and honest uncertainty. It reuses existing cases and the context-efficiency measurement contract. Implementation is complete; no concrete comparison candidate warranted execution, no comparison was run and measured effectiveness remains unestablished.

On-demand lesson discovery

SAVED: demir-bot/references/capability-maintenance.md and routes.md now cover on-request analysis of selected authorized history, evidence and counterexamples, source provenance, redaction and inactive lesson proposals. Source content remains untrusted. Reusable lessons require explicit persistence scope and removal of project-specific private data before promotion. Existing learning controls and maintenance routes are reused; no new specialist or runner was created. Guidance implementation is complete; no selected-history discovery or lesson promotion was performed.

All paths in this section are package-relative unless otherwise stated. Private profiles, credentials, runtime paths and rollback contents are excluded. No measured savings are established. The latest installed snapshot includes this saved guidance; installation equality does not establish execution behavior.

New priority order and decision gates

A1 general guidance, including on-demand lesson discovery, is saved. Behavioral regression is complete under the user's accepted ten-case scope, and bounded-comparison guidance is saved. A2 is the next remaining batch: implement only project-relevant engineering gaps with a named project and actual evidence. A3 is optional. B resolves combined-workflow conflicts and provider boundaries; C covers remaining release-level evidence; D covers distribution and lifecycle; E covers final documentation and release decisions; F remains inactive or reuse-only. Completed guidance is reused rather than reopened.

Focused regression checkpoints follow each meaningful authorized implementation batch in A and B, especially permission, privacy and routing changes. Reuse the accepted behavioral cases, Structural repository checks and the existing QA evaluation reference: source checks first, small independent synthetic cases where explicitly authorized. Case counts, execution limits and retries must be declared; no open-ended fix/evaluate loop. This schedule does not grant execution permission. Minimal dependency facts, acceptance criteria and rollback notes are maintained while implementing; polished guides wait until E.

No artificial dependency requires every A3 item before B/C. Select the useful release scope, record exclusions and validate that scope. Licensing/provenance notices and privacy boundaries apply from the first edit; final redistribution review belongs in D. CI configuration may be drafted when checks stabilize but enabling remote execution needs its own authority. Optional app deployment skills do not authorize deploying the plugin.

Overlap resolution: 001/004/003 share Swift/architecture ownership; 006 extends Error and type design error contracts; 019/020/021/022/029/030/044 extend QA instead of duplicate test coordinators; 018 owns profiling depth; 036 owns typed connector contracts; User-directed learning, Private lesson register, Controlled skill evolution, Bounded improvement comparisons and effectiveness, Learning controls and correction, On-demand lesson discovery reuse maintenance/privacy; 089 recipes and Broader GitHub operations operations remain distinct. 099/100/111/112 are explicit reuse decisions, not missing specialists. Keep distinct acceptance outcomes when extending shared owners.

A2 - Project-led engineering depth

001. Swift concurrency, state ownership and identity

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Extend the existing SwiftUI specialist with a concise source-review entrypoint for isolation, Sendable, task lifetime and cancellation; no duplicate standalone skill unless justified. Extend existing isolation guidance using the actual Swift language mode, compiler and target flags; address diagnostics without blanket unsafe suppressions. Reuse structured concurrency, cooperative cancellation and actor guidance; add only missing project-relevant async sequences, continuations or task-group patterns. Reuse existing ownership and observation guidance; add only an uncovered state-lifetime or dependency-injection case with deployment-target compatibility. Reuse stable identity guidance for lists, navigation and task lifetime; address an actual identity/reset failure without a duplicate specialist. Extend the existing SwiftUI specialist with a concise source-review entrypoint for isolation, Sendable, task lifetime and cancellation; no duplicate standalone skill unless justified.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Extend the existing SwiftUI specialist only for a demonstrated missing case; do not create five overlapping skills.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 001, Swift concurrency, state ownership and identity. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Extend the existing SwiftUI specialist only for a demonstrated missing case; do not create five overlapping skills. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

002. ios-project-structure

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Map relevant targets, schemes, packages, resources and configuration for a named iOS project; keep a small useful navigation reference rather than loading a full repository map each turn.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 002, ios-project-structure. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Map relevant targets, schemes, packages, resources and configuration for a named iOS project; keep a small useful navigation reference rather than loading a full repository map each turn. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

003. ios-architecture-patterns

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Apply existing architecture review to iOS state/data boundaries and modularity; choose a pattern from actual requirements rather than enforcing one architecture everywhere.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 003, ios-architecture-patterns. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Apply existing architecture review to iOS state/data boundaries and modularity; choose a pattern from actual requirements rather than enforcing one architecture everywhere. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

004. dependency-injection-ios

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Extend architecture guidance with stable service ownership and replaceable dependencies using project conventions; do not introduce a framework by default.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 004, dependency-injection-ios. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Extend architecture guidance with stable service ownership and replaceable dependencies using project conventions; do not introduce a framework by default. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

005. swiftui-navigation

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add deployment-aware navigation state, routes, restoration and deep-link handling for the actual app architecture.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 005, swiftui-navigation. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add deployment-aware navigation state, routes, restoration and deep-link handling for the actual app architecture. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

006. ios-error-handling

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Define typed failure, cancellation, recovery and user messaging across the actual iOS boundaries; preserve error information and avoid silent success fallbacks.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 006, ios-error-handling. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Define typed failure, cancellation, recovery and user messaging across the actual iOS boundaries; preserve error information and avoid silent success fallbacks. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

007. xcodebuild-error-taxonomy

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Classify supplied Xcode build diagnostics into compiler, linker, dependency, signing and environment causes; investigate the actual failure without automatically rerunning builds.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 007, xcodebuild-error-taxonomy. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Classify supplied Xcode build diagnostics into compiler, linker, dependency, signing and environment causes; investigate the actual failure without automatically rerunning builds. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

008. derived-data-hygiene

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Diagnose build-cache anomalies and choose scoped recovery; preserve working caches and do not delete DerivedData as a routine first step.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 008, derived-data-hygiene. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Diagnose build-cache anomalies and choose scoped recovery; preserve working caches and do not delete DerivedData as a routine first step. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

009. spm-dependency-management

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Handle package versions, resolution, resources and binary targets within the project's supported toolchain; preserve lockfiles and review supply-chain implications.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 009, spm-dependency-management. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Handle package versions, resolution, resources and binary targets within the project's supported toolchain; preserve lockfiles and review supply-chain implications. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

010. keychain-cryptokit

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add platform-correct secret storage and cryptographic API guidance; distinguish access groups, availability and backup behavior without inventing cryptography.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 010, keychain-cryptokit. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add platform-correct secret storage and cryptographic API guidance; distinguish access groups, availability and backup behavior without inventing cryptography. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

011. swiftdata-migrations

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Design versioned SwiftData schemas and migration paths with recovery assumptions; never migrate live user data or run a migration without explicit scope.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Named SwiftData schema versions, sample synthetic data and recovery design.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 011, swiftdata-migrations. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Design versioned SwiftData schemas and migration paths with recovery assumptions; never migrate live user data or run a migration without explicit scope. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: Named SwiftData schema versions, sample synthetic data and recovery design. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

012. coredata-to-swiftdata

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Plan a project-specific store transition with mapping, relationships, compatibility and rollback; establish data ownership and preserve existing stores.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: 011 plus actual Core Data mapping and supported toolchain; never migrate live data incidentally.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 012, coredata-to-swiftdata. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Plan a project-specific store transition with mapping, relationships, compatibility and rollback; establish data ownership and preserve existing stores. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: 011 plus actual Core Data mapping and supported toolchain; never migrate live data incidentally. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

013. cloudkit-sync

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Design ownership, conflict resolution, offline behavior and schema environments for a concrete CloudKit app; do not alter production containers incidentally.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 013, cloudkit-sync. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Design ownership, conflict resolution, offline behavior and schema environments for a concrete CloudKit app; do not alter production containers incidentally. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

014. ios-accessibility-audit

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Review VoiceOver, Dynamic Type, focus, contrast and motion for the actual iOS interface; distinguish source review from device-tested accessibility.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 014, ios-accessibility-audit. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Review VoiceOver, Dynamic Type, focus, contrast and motion for the actual iOS interface; distinguish source review from device-tested accessibility. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

015. ios-localization

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Handle string catalogs, pluralization, locale-sensitive formats and right-to-left layouts with approved translations and deployment compatibility.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 015, ios-localization. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Handle string catalogs, pluralization, locale-sensitive formats and right-to-left layouts with approved translations and deployment compatibility. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

016. privacy-manifests

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Inspect actual SDK/data behavior and current Apple requirements before preparing privacy declarations; never infer compliance from a template alone.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 016, privacy-manifests. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Inspect actual SDK/data behavior and current Apple requirements before preparing privacy declarations; never infer compliance from a template alone. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

017. ios-security-hardening

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Reuse Codex Security and privacy routes first; add iOS-specific transport, storage, entitlement and IPC guidance for demonstrated gaps.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 017, ios-security-hardening. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Reuse Codex Security and privacy routes first; add iOS-specific transport, storage, entitlement and IPC guidance for demonstrated gaps. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

018. Performance analysis and Apple profiling depth

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use existing architecture/release/SwiftUI expertise first; add workload-specific depth only when needed. Latency, throughput, memory, I/O, hot-path analysis and representative workloads. Profiling and benchmarks remain opt-in. Interpret supplied traces or prepare an explicitly requested profiling workflow with a representative workload; distinguish hypotheses from measured bottlenecks. Investigate startup dependencies, synchronous work and first useful frame using actual evidence; preserve correctness and report device/build conditions for any measurement. Investigate ownership cycles, retained tasks and memory growth from source or supplied graphs; device profiling remains opt-in. Connect supported trace capture/reading for a named workload only when profiling is authorized; use roadmap item 018 guidance and preserve private trace data.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Reuse architecture/SwiftUI guidance; add workload-specific Instruments, memory or launch guidance and tooling only for a named project and authorized profiling.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 018, Performance analysis and Apple profiling depth. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Reuse architecture/SwiftUI guidance; add workload-specific Instruments, memory or launch guidance and tooling only for a named project and authorized profiling. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

019. xctest-patterns

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add focused XCTest guidance to existing QA for fixtures, async behavior and meaningful assertions; separate writing tests from executing them.

Reuse: qa-and-test-evidence, swiftui-performance-and-concurrency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 019, xctest-patterns. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add focused XCTest guidance to existing QA for fixtures, async behavior and meaningful assertions; separate writing tests from executing them. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

020. swift-testing-patterns

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add current Swift Testing guidance for parameterization, traits and concurrency when supported by the project; do not migrate XCTest merely for novelty.

Reuse: qa-and-test-evidence, swiftui-performance-and-concurrency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 020, swift-testing-patterns. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add current Swift Testing guidance for parameterization, traits and concurrency when supported by the project; do not migrate XCTest merely for novelty. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

021. ui-test-patterns

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add focused iOS UI test patterns for stable identifiers, asynchronous states and isolation; avoid broad UI campaigns unless requested.

Reuse: qa-and-test-evidence, swiftui-performance-and-concurrency. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 021, ui-test-patterns. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add focused iOS UI test patterns for stable identifiers, asynchronous states and isolation; avoid broad UI campaigns unless requested. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

022. snapshot-testing

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add visual-test guidance only for an actual regression risk; control device, OS, font and rendering variability, with explicit baseline approval.

Reuse: qa-and-test-evidence, swiftui-performance-and-concurrency. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 022, snapshot-testing. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add visual-test guidance only for an actual regression risk; control device, OS, font and rendering variability, with explicit baseline approval. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

023. swiftlint-swiftformat-ios

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use the project's existing Swift lint/format configuration on request; distinguish style from correctness and avoid automatic save hooks or whole-repository reformatting.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 023, swiftlint-swiftformat-ios. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Use the project's existing Swift lint/format configuration on request; distinguish style from correctness and avoid automatic save hooks or whole-repository reformatting. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

024. simulator-device-management

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use supported tools for requested simulator/device selection, launching and screenshots; identify the exact target and preserve unrelated devices/data.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 024, simulator-device-management. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Use supported tools for requested simulator/device selection, launching and screenshots; identify the exact target and preserve unrelated devices/data. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

025. Xcode CLI integration

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Discover supported installed Xcode command-line tools and exact contracts for a requested operation; verify current upload tooling rather than assuming historical altool examples still apply.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 025, Xcode CLI integration. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Discover supported installed Xcode command-line tools and exact contracts for a requested operation; verify current upload tooling rather than assuming historical altool examples still apply. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

026. Backend and API engineering

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use existing architecture, requirements, Superpowers engineering workflows and official stack guidance first. Add narrow references for contracts, authorization, validation, pagination, idempotency, jobs, caching or rate limits only when demonstrated gaps justify them. Superpowers is a methodology, not guaranteed backend expertise.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Existing architecture and Superpowers coverage is reused, and the contract has concrete consumers and observable acceptance criteria.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 026, Backend and API engineering. Reuse the existing coordinator routes and relevant available specialist/provider first. Use existing architecture, requirements, Superpowers engineering workflows and official stack guidance first. Add narrow references for contracts, authorization, validation, pagination, idempotency, jobs, caching or rate limits only when demonstrated gaps justify them. Superpowers is a methodology, not guaranteed backend expertise. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

027. Database engineering

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use the existing Supabase/Postgres provider for applicable schema, migration, RLS and query work. Add database-specific guidance only outside that coverage or after a concrete failure. Provider installation is not proof of account access, restore safety or successful migrations.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: No production migration or restore is implied. New guidance names the database, compatibility constraints and recovery assumptions.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 027, Database engineering. Reuse the existing coordinator routes and relevant available specialist/provider first. Use the existing Supabase/Postgres provider for applicable schema, migration, RLS and query work. Add database-specific guidance only outside that coverage or after a concrete failure. Provider installation is not proof of account access, restore safety or successful migrations. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

028. React/frontend depth

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Reuse frontend-quality-and-accessibility, Figma, Sites and relevant Superpowers workflows. Add React state, data fetching, rendering, forms and performance references only for a real uncovered need. Design/hosting tools are not substitutes for all framework knowledge.

Reuse: frontend-quality-and-accessibility. Canonical shared references:
plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;
plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: The result preserves accessibility and existing conventions; a design provider is not mistaken for framework-runtime evidence.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 028, React/frontend depth. Reuse the existing coordinator routes and relevant available specialist/provider first. Reuse frontend-quality-and-accessibility, Figma, Sites and relevant Superpowers workflows. Add React state, data fetching, rendering, forms and performance references only for a real uncovered need. Design/hosting tools are not substitutes for all framework knowledge. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse frontend-quality-and-accessibility and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

029. Stack-specific testing references

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use existing QA guidance and relevant Superpowers testing workflows first. Add pytest, JavaScript/TypeScript or React recipes only when a real project reveals missing practical detail. Preserve explicit test scope and avoid universal coverage targets.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: The recipe fits the actual framework and tests observable behavior; no duplicate testing coordinator or mandatory coverage percentage.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 029, Stack-specific testing references. Reuse qa-and-test-evidence and its evaluation reference first. Use existing QA guidance and relevant Superpowers testing workflows first. Add pytest, JavaScript/TypeScript or React recipes only when a real project reveals missing practical detail. Preserve explicit test scope and avoid universal coverage targets. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

030. Project E2E and accessibility testing

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Reuse QA, frontend accessibility and available browser tooling. Add stack-specific setup, traces and flaky-test recovery only for real project gaps. Superpowers does not itself establish working browser access or complete accessibility coverage. Demir Bot regression coverage accompanies implementation batches and receives comprehensive validation in phase C.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Selected user flows and error paths are observable. Automated accessibility checks are not claimed to prove full conformance.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 030, Project E2E and accessibility testing. Reuse qa-and-test-evidence and its evaluation reference first. Reuse QA, frontend accessibility and available browser tooling. Add stack-specific setup, traces and flaky-test recovery only for real project gaps. Superpowers does not itself establish working browser access or complete accessibility coverage. Demir Bot regression coverage accompanies implementation batches and receives comprehensive validation in phase C. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

031. UI design systems

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use Figma and existing frontend routes first. Tokens, components, consistency and responsive states through existing Figma/frontend routes; adapt ECC design-system and interface-polish guidance selectively.

Reuse: frontend-quality-and-accessibility. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: The local addition fills a demonstrated gap and does not replace provider design tooling.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 031, UI design systems. Reuse the existing coordinator routes and relevant available specialist/provider first. Use Figma and existing frontend routes first. Tokens, components, consistency and responsive states through existing Figma/frontend routes; adapt ECC design-system and interface-polish guidance selectively. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse frontend-quality-and-accessibility and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

032. Interaction and click-path reviews

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use existing frontend/QA routes first. Review complete task flows, empty/error/loading states, keyboard use and mobile usability. Browser execution requires the applicable scope.

Reuse: frontend-quality-and-accessibility. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Findings are tied to observable task failures; browser checks are scoped rather than automatically attached to every edit.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 032, Interaction and click-path reviews. Reuse the existing coordinator routes and relevant available specialist/provider first. Use existing frontend/QA routes first. Review complete task flows, empty/error/loading states, keyboard use and mobile usability. Browser execution requires the applicable scope. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse frontend-quality-and-accessibility and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

033. Containers and reproducible environments

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add only for projects that actually use containers. Development versus production containers, pinned dependencies, environment configuration, secrets and clean teardown. No blanket Docker requirement for an instruction plugin.

Reuse: release-readiness-and-observability. Canonical shared references:
plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;
plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: The environment is reproducible within stated support; no container infrastructure is added to the instruction-only plugin without need.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 033, Containers and reproducible environments. Reuse the existing coordinator routes and relevant available specialist/provider first. Add only for projects that actually use containers. Development versus production containers, pinned dependencies, environment configuration, secrets and clean teardown. No blanket Docker requirement for an instruction plugin. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse release-readiness-and-observability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

034. Operational observability

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Reuse existing release-readiness guidance and hosting tools first. Useful structured events, error correlation, latency, health and retention. Keep private payloads out of logs and dashboards.

Reuse: release-readiness-and-observability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Telemetry has clear owners, retention and redaction; alerts or background collectors are not enabled incidentally.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 034, Operational observability. Reuse the existing coordinator routes and relevant available specialist/provider first. Reuse existing release-readiness guidance and hosting tools first. Useful structured events, error correlation, latency, health and retention. Keep private payloads out of logs and dashboards. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse release-readiness-and-observability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

035. Model/API cost control

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Current pricing sources, explicit budgets, narrow retries, caching and routing trade-offs for custom API applications. Do not promise subscription savings or override the user's selected model.

Reuse: context-efficiency, architecture-review. Canonical shared references:
plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;
plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Estimated cost is labelled; no model change or purchase is automatic and Plus allowance is not treated as an API budget.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 035, Model/API cost control. Reuse the existing coordinator routes and relevant available specialist/provider first. Current pricing sources, explicit budgets, narrow retries, caching and routing trade-offs for custom API applications. Do not promise subscription savings or override the user's selected model. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse context-efficiency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

036. Typed connector development and App Store Connect

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Reuse OpenAI Developers and suitable provider SDK/tooling first. Typed contracts, pagination, auth, secrets, retries, cancellation, idempotency and precise read/write effects. Reuse official SDK/provider guidance. Define supported typed operations and minimal permissions for the actual App Store Connect task; distinguish reads, metadata edits, uploads and submission. Implement an authenticated, narrowly scoped operation after defining the item 036 contract; keep credentials outside the package and distinguish accepted jobs from completed actions.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Named provider operation, official SDK contract and available supported access; this item has no self-dependency.

Observable acceptance: Reuse existing SDK/provider coverage; implement a narrow operation only after the target, credentials boundary and read/write scope are known.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 036, Typed connector development and App Store Connect. Reuse the existing coordinator routes and relevant available specialist/provider first. Reuse existing SDK/provider coverage; implement a narrow operation only after the target, credentials boundary and read/write scope are known. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: Named provider operation, official SDK contract and available supported access; this item has no self-dependency. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

037. code-signing-provisioning

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Diagnose identities, entitlements, bundle IDs and provisioning for a named target; do not change account access, certificates or signing assets without applicable authorization.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 037, code-signing-provisioning. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Diagnose identities, entitlements, bundle IDs and provisioning for a named target; do not change account access, certificates or signing assets without applicable authorization. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

038. app-store-review-compliance

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Review a named app against current official submission requirements; document uncertainty and avoid guarantees of acceptance.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 038, app-store-review-compliance. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Review a named app against current official submission requirements; document uncertainty and avoid guarantees of acceptance. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

039. testflight-automation

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Prepare an explicitly requested TestFlight workflow with versioning, signing, release notes and upload evidence; actual upload/distribution requires its own scope.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: 037 signing, 038 submission constraints and 025/036 supported operations; upload requires separate authority.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 039, testflight-automation. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Prepare an explicitly requested TestFlight workflow with versioning, signing, release notes and upload evidence; actual upload/distribution requires its own scope. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: 037 signing, 038 submission constraints and 025/036 supported operations; upload requires separate authority. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

A3 - Optional capabilities when a real need exists

040. Data and ML workflow depth

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Reuse the ML specialist and project frameworks; add only demonstrated gaps. Data quality, leakage, reproducibility, dataset/model lineage, fair baselines, inference and throughput. Training and paid compute remain separately authorized.

Reuse: ml-training-specialist, swiftui-performance-and-concurrency. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Training, data transfer and compute spending remain separate; baselines and evaluation leakage are explicit.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 040, Data and ML workflow depth. Reuse the existing coordinator routes and relevant available specialist/provider first. Reuse the ML specialist and project frameworks; add only demonstrated gaps. Data quality, leakage, reproducibility, dataset/model lineage, fair baselines, inference and throughput. Training and paid compute remain separately authorized. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse ml-training-specialist, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

041. Cross-client memory

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Explicit scopes, provenance, corrections, retention and deletion. Evaluate ECC unified-memory as a separate runtime with maintenance cost.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Private/user scopes never become global implicitly; memory cannot supply authorization and raw conversations are not retained by default.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 041, Cross-client memory. Reuse the existing coordinator routes and relevant available specialist/provider first. Explicit scopes, provenance, corrections, retention and deletion. Evaluate ECC unified-memory as a separate runtime with maintenance cost. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

042. Local status dashboard

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Read-only view of skills, versions, provider states and approval status. Avoid exposing private profile data; a dashboard is not required for ordinary use.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: The display distinguishes stale/unknown evidence and never exposes profile contents, secrets or implied verified access.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 042, Local status dashboard. Reuse the existing coordinator routes and relevant available specialist/provider first. Read-only view of skills, versions, provider states and approval status. Avoid exposing private profile data; a dashboard is not required for ordinary use. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

043. Notification integrations

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: User-requested notifications with deduplication and minimal content. Reuse providers; no background notification service by default.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: No unsolicited watcher or messaging is enabled; unchanged/non-actionable state stays quiet when monitoring is authorized.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 043, Notification integrations. Reuse the existing coordinator routes and relevant available specialist/provider first. User-requested notifications with deduplication and minimal content. Reuse providers; no background notification service by default. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

044. Advanced testing modes

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Visual regression, Windows desktop E2E, fuzzing, mutation, load and fault injection when the project risk justifies them. Existing QA already discusses several techniques.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Contain execution, define recovery and record limits; broad testing is not a default prerequisite for ordinary edits.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 044, Advanced testing modes. Reuse qa-and-test-evidence and its evaluation reference first. Visual regression, Windows desktop E2E, fuzzing, mutation, load and fault injection when the project risk justifies them. Existing QA already discusses several techniques. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

045. Additional language/framework packs

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Go, Rust, Java/Spring/JPA/Quarkus, Kotlin/Ktor/Exposed, C++/C#/F#, PHP/Laravel, Ruby/Rails, Perl, Django/Celery, NestJS, Bun, Vue/Nuxt, Angular and Vite/Next tooling. Add only the active stack.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: The added guidance is version-aware and nonduplicative; this item does not authorize installing every listed stack.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 045, Additional language/framework packs. Reuse the existing coordinator routes and relevant available specialist/provider first. Go, Rust, Java/Spring/JPA/Quarkus, Kotlin/Ktor/Exposed, C++/C#/F#, PHP/Laravel, Ruby/Rails, Perl, Django/Celery, NestJS, Bun, Vue/Nuxt, Angular and Vite/Next tooling. Add only the active stack. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

046. Mobile and on-device extensions

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Android, React Native, Flutter, Compose Multiplatform, Swift persistence/testing and on-device models. Retain existing SwiftUI coverage.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Document device/runtime limitations and test scope; no claim that one mobile pack supports every platform.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 046, Mobile and on-device extensions. Reuse the existing coordinator routes and relevant available specialist/provider first. Android, React Native, Flutter, Compose Multiplatform, Swift persistence/testing and on-device models. Retain existing SwiftUI coverage. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

047. Specialist infrastructure

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Redis, MySQL, ClickHouse, Kubernetes, alternative hosting/environment tools and workload-specific databases. Add when used by an actual project.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Operational ownership, auth, rollback and cost are concrete; no provisioning or cloud spending is implied.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 047, Specialist infrastructure. Reuse the existing coordinator routes and relevant available specialist/provider first. Redis, MySQL, ClickHouse, Kubernetes, alternative hosting/environment tools and workload-specific databases. Add when used by an actual project. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

048. SEO and broader marketing

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Search visibility, campaign planning, competitive reports and analytics. Keep unverified performance claims out of public copy.

Reuse: personal-writing, demir-linkedin, deep-research-and-idea-validation. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: No guaranteed traffic, ranking or conversion claims; publishing and analytics access remain separate.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 048, SEO and broader marketing. Reuse the existing coordinator routes and relevant available specialist/provider first. Search visibility, campaign planning, competitive reports and analytics. Keep unverified performance claims out of public copy. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse personal-writing, demir-linkedin, deep-research-and-idea-validation and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

049. Networking and opportunity research

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Connections, lead research and social-graph ideas where lawful and useful. Use minimal data; outreach and profile changes need explicit scope.

Reuse: personal-writing, demir-linkedin, deep-research-and-idea-validation. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: No sensitive profiling or unapproved outreach; identity and recipient resolution precede any separately authorized contact.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 049, Networking and opportunity research. Reuse the existing coordinator routes and relevant available specialist/provider first. Connections, lead research and social-graph ideas where lawful and useful. Use minimal data; outreach and profile changes need explicit scope. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse personal-writing, demir-linkedin, deep-research-and-idea-validation and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

050. Investor/business documents

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Pitch materials and outreach drafts for an actual venture. Reuse writing/research; sending and commercial claims remain separately controlled.

Reuse: personal-writing, demir-linkedin, deep-research-and-idea-validation. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: No invented traction, revenue or investor contact; outreach remains a separate approved action.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 050, Investor/business documents. Reuse the existing coordinator routes and relevant available specialist/provider first. Pitch materials and outreach drafts for an actual venture. Reuse writing/research; sending and commercial claims remain separately controlled. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse personal-writing, demir-linkedin, deep-research-and-idea-validation and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

051. Extended social channels

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Cross-platform drafts and X or other publishing providers when requested. Account access and approval do not transfer automatically between channels.

Reuse: personal-writing, demir-linkedin, deep-research-and-idea-validation. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Reuse and publication permissions are distinct; one channel authorization does not transfer to another.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 051, Extended social channels. Reuse the existing coordinator routes and relevant available specialist/provider first. Cross-platform drafts and X or other publishing providers when requested. Account access and approval do not transfer automatically between channels. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse personal-writing, demir-linkedin, deep-research-and-idea-validation and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

052. Media and motion extras

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Manim, Remotion, Blender inspection, VideoDB, motion recipes, UI demos, icon generation and visual taste calibration. Prefer existing media/design providers before installing alternatives.

Reuse: demir-linkedin, frontend-quality-and-accessibility. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Asset rights, consent, cost and export requirements are explicit; no generation charge or external upload from guidance alone.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 052, Media and motion extras. Reuse the existing coordinator routes and relevant available specialist/provider first. Manim, Remotion, Blender inspection, VideoDB, motion recipes, UI demos, icon generation and visual taste calibration. Prefer existing media/design providers before installing alternatives. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-linkedin, frontend-quality-and-accessibility and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

053. Specialized research sources

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: PubMed, USPTO, scientific packages and other domain databases only for suitable research. Ordinary technical research does not need all of them.

Reuse: deep-research-and-idea-validation, jurisdiction-specific-legal-feasibility. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Record search scope, access limits and citation support; domain tools are not installed as generic research prerequisites.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 053, Specialized research sources. Reuse the existing coordinator routes and relevant available specialist/provider first. PubMed, USPTO, scientific packages and other domain databases only for suitable research. Ordinary technical research does not need all of them. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse deep-research-and-idea-validation, jurisdiction-specific-legal-feasibility and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

054. Workspace and messaging connectors

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Jira, messaging, mail and additional workspace integrations when needed. Existing Google/document tools cover much of this; inspect actual available access.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: New account connections or permissions require the appropriate user action; provider existence is not authentication.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 054, Workspace and messaging connectors. Reuse the existing coordinator routes and relevant available specialist/provider first. Jira, messaging, mail and additional workspace integrations when needed. Existing Google/document tools cover much of this; inspect actual available access. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

055. widgetkit-live-activities

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add target-aware WidgetKit and Live Activity guidance covering shared data, timelines, lifecycle and platform constraints.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 055, widgetkit-live-activities. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add target-aware WidgetKit and Live Activity guidance covering shared data, timelines, lifecycle and platform constraints. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

056. app-intents-shortcuts

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Design actual App Intents and Shortcuts contracts with clear parameters, privacy boundaries and supported invocation behavior.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 056, app-intents-shortcuts. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Design actual App Intents and Shortcuts contracts with clear parameters, privacy boundaries and supported invocation behavior. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

057. background-tasks

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Choose supported iOS background execution modes for a real requirement and handle expiration/cancellation; do not promise exact scheduling.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 057, background-tasks. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Choose supported iOS background execution modes for a real requirement and handle expiration/cancellation; do not promise exact scheduling. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

058. push-notifications

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Define permission, device-token lifecycle, environments and payload handling; do not send notifications or provision credentials from guidance alone.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 058, push-notifications. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Define permission, device-token lifecycle, environments and payload handling; do not send notifications or provision credentials from guidance alone. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

059. universal-links-deep-linking

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Implement validated route parsing, association requirements and safe fallback for authorized app/domain ownership.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 059, universal-links-deep-linking. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Implement validated route parsing, association requirements and safe fallback for authorized app/domain ownership. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

060. storekit2-subscriptions

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Design entitlement and transaction-state handling for a named product using current StoreKit guidance; purchases, account setup and commercial commitments remain separate.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 060, storekit2-subscriptions. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Design entitlement and transaction-state handling for a named product using current StoreKit guidance; purchases, account setup and commercial commitments remain separate. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

061. coreml-on-device

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Assess model compatibility, device resources, preprocessing and privacy for an actual Core ML feature; do not start training or acquire models incidentally.

Reuse: ml-training-specialist, swiftui-performance-and-concurrency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 061, coreml-on-device. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Assess model compatibility, device resources, preprocessing and privacy for an actual Core ML feature; do not start training or acquire models incidentally. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse ml-training-specialist, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

062. foundation-models-ios

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use current official on-device model APIs only on supported devices/OS versions; specify availability, fallback, privacy and task limitations.

Reuse: ml-training-specialist, swiftui-performance-and-concurrency. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 062, foundation-models-ios. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Use current official on-device model APIs only on supported devices/OS versions; specify availability, fallback, privacy and task limitations. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse ml-training-specialist, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

063. vision-speech-frameworks

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Select relevant Vision or Speech APIs for a concrete task; handle permissions, supported languages, accuracy and on-device/cloud behavior explicitly.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 063, vision-speech-frameworks. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Select relevant Vision or Speech APIs for a concrete task; handle permissions, supported languages, accuracy and on-device/cloud behavior explicitly. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

064. metal-basics

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add narrow GPU rendering/compute guidance for a demonstrated workload; retain resource lifetime and device compatibility constraints.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 064, metal-basics. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add narrow GPU rendering/compute guidance for a demonstrated workload; retain resource lifetime and device compatibility constraints. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

065. avfoundation-media

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Add scoped capture/playback/export guidance with permissions, session lifecycle, interruption and media ownership handling.

Reuse: swiftui-performance-and-concurrency, architecture-review. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 065, avfoundation-media. Reuse swiftui-performance-and-concurrency and relevant QA, architecture, privacy or release skills first. Add scoped capture/playback/export guidance with permissions, session lifecycle, interruption and media ownership handling. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse swiftui-performance-and-concurrency, architecture-review and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

066. Fastlane integration, optional

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use the project's existing Fastlane setup only when relevant; preserve signing secrets and require explicit scope for uploads, account writes or release actions.

Reuse: release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 066, Fastlane integration, optional. Reuse the existing coordinator routes and relevant available specialist/provider first. Use the project's existing Fastlane setup only when relevant; preserve signing secrets and require explicit scope for uploads, account writes or release actions. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse release-readiness-and-observability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

067. Xcode Cloud integration, optional

Status: CONDITIONAL; implement only for an actual useful project gap.

Purpose and remaining deliverable: Use an existing authorized Xcode Cloud project only when needed; define build triggers, secrets, costs and release boundaries without enabling background jobs by default.

Reuse: release-readiness-and-observability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Useful capability before distribution; reuse source coverage and gate project-specific operations.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 067, Xcode Cloud integration, optional. Reuse the existing coordinator routes and relevant available specialist/provider first. Use an existing authorized Xcode Cloud project only when needed; define build triggers, secrets, costs and release boundaries without enabling background jobs by default. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse release-readiness-and-observability and the references named in this entry. Prerequisites: A named project, actual stack/tool versions, supplied evidence and operation-specific access are needed for project implementation; do not invent these dependencies. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

B - Integrate and refine

068. Action receipts and selective executable safeguards

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: For custom runtimes: bind permission to exact action/content/destination, reuse valid approval, invalidate material changes and reconcile uncertain outcomes. Native provider permissions still apply. Use native permissions first; consider narrow compatible guards for custom runtime actions. Review bypasses and failure modes; do not import Claude hooks as if they enforce Codex behavior.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A named custom runtime and real per-operation contracts; native permissions first.

Observable acceptance: Bind any custom runtime approval to the exact action and result; source guidance alone does not enforce authorization.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Resolve cross-skill contracts before comprehensive validation.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 068, Action receipts and selective executable safeguards. Reuse the existing coordinator routes and relevant available specialist/provider first. Bind any custom runtime approval to the exact action and result; source guidance alone does not enforce authorization. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: A named custom runtime and real per-operation contracts; native permissions first. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

069. Agent/runtime diagnostics

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Keep agent-review guidance focused on observed runtime failures. Inspect hidden model calls, stale memory, tool contracts, retries and transport/output changes. Distinguish instruction failures from unavailable tools or platform limits.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Diagnosis identifies the failed layer and contained recovery; it does not add unrequested agents, retry loops or global interception.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Resolve cross-skill contracts before comprehensive validation.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 069, Agent/runtime diagnostics. Reuse the existing coordinator routes and relevant available specialist/provider first. Keep agent-review guidance focused on observed runtime failures. Inspect hidden model calls, stale memory, tool contracts, retries and transport/output changes. Distinguish instruction failures from unavailable tools or platform limits. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

070. Upstream maintenance tooling

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: On-demand version fingerprints, adaptation diffs and keep/update/retire decisions. Preserve intentional local differences and license notices. Identify redundant or ineffective skills from authorized evidence and propose merge/retirement; do not delete installed skills automatically. Suggest retirement only during a requested review using actual evidence; no automatic background scoring, mutation or removal.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Updates preserve local behavior; no watcher, automatic upstream import or indiscriminate full re-audit.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Resolve cross-skill contracts before comprehensive validation.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 070, Upstream maintenance tooling. Reuse the existing coordinator routes and relevant available specialist/provider first. On-demand version fingerprints, adaptation diffs and keep/update/retire decisions. Preserve intentional local differences and license notices. Identify redundant or ineffective skills from authorized evidence and propose merge/retirement; do not delete installed skills automatically. Suggest retirement only during a requested review using actual evidence; no automatic background scoring, mutation or removal. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

071. Bounded fresh-context review

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Define one explicitly requested independent review with only the diff, acceptance criteria and necessary context; return findings, not fixes. Avoid default delegation and measure coordination cost. Match review scope to risk and user request; define quick, focused and release review without making review automatic. Define one explicitly requested independent review with only the diff, acceptance criteria and necessary context; return findings, not fixes. Avoid default delegation and measure coordination cost.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Deliver the scoped outcome above with named dependencies and an explicit acceptance example. For instructions, identify the trigger, expected action and no-action case; for tooling, identify the operation, expected result and failure/recovery behavior. Mark operational evidence unverified until observed within authorized scope.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Resolve cross-skill contracts before comprehensive validation.

Coordination decision: use at most one coordinator-to-worker layer in an explicitly authorized bounded review; preserve independent evidence and parent acceptance. No recursive delegation or automatic reviewers.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 071, Bounded fresh-context review. Reuse the existing coordinator routes and relevant available specialist/provider first. Define one explicitly requested independent review with only the diff, acceptance criteria and necessary context; return findings, not fixes. Avoid default delegation and measure coordination cost. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior. Coordination decision: use at most one coordinator-to-worker layer in an explicitly authorized bounded review; preserve independent evidence and parent acceptance. No recursive delegation or automatic reviewers.

072. Opt-in gate tiers and skip policy

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Define optional risk-based checks for actual projects. Replace the continuation's automatic save/commit defaults with explicitly authorized execution; preserve applicable release requirements. Define inexpensive, medium and release-level checks for an actual project; all execution remains explicitly opt-in. Do not install automatic on-save or on-commit hooks by default.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: No default on-save/on-commit execution. Each gate has an authorized trigger, meaningful risk, bounded cost and skip rule.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Resolve cross-skill contracts before comprehensive validation.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 072, Opt-in gate tiers and skip policy. Reuse the existing coordinator routes and relevant available specialist/provider first. No default on-save/on-commit execution. Each gate has an authorized trigger, meaningful risk, bounded cost and skip rule. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

073. Documentation maintenance

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Clear ownership, source-of-truth rules, stale claims and consistent installation/catalogue examples. Avoid generating parallel documents that drift.

Reuse: documentation-and-adrs. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Update the smallest relevant documents and remove genuine duplication without losing material requirements.

Verification stage: Focused source review after implementation; explicitly authorized batch regressions in A/B, comprehensive evidence in C.

Ordering reason: Resolve cross-skill contracts before comprehensive validation.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 073, Documentation maintenance. Reuse documentation-and-adrs and the current README/manifest first. Clear ownership, source-of-truth rules, stale claims and consistent installation/catalogue examples. Avoid generating parallel documents that drift. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse documentation-and-adrs and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

C - Comprehensive validation

074. Confidentiality release gate

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Review outgoing files, examples, generated artifacts, metadata and relevant history. Keep private exclusion rules outside Git. Renaming alone is insufficient; remote cleanup and history rewriting require scoped authorization.

Reuse: privacy-review, agent-configuration-review, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Known exclusions are checked without embedding them in public tests or logs. A name replacement alone cannot satisfy the gate.

Verification stage: Explicitly authorized comprehensive structural/behavioral/privacy evidence; no implied model runs.

Ordering reason: Establish evidence for the selected scope before distribution.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 074, Confidentiality release gate. Reuse the existing coordinator routes and relevant available specialist/provider first. Review outgoing files, examples, generated artifacts, metadata and relevant history. Keep private exclusion rules outside Git. Renaming alone is insufficient; remote cleanup and history rewriting require scoped authorization. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse privacy-review, agent-configuration-review, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

D - Packaging and installation lifecycle

075. Dependency and support matrix

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Required versus optional tools, client/OS support, supported versions, accounts and costs. Distinguish installed, exposed, authenticated and operation-tested.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Evidence includes environment and operation scope; historical success is not promoted to current access or all-operation verification.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 075, Dependency and support matrix. Reuse documentation-and-adrs and the current README/manifest first. Required versus optional tools, client/OS support, supported versions, accounts and costs. Distinguish installed, exposed, authenticated and operation-tested. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

076. Supply-chain records

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Dependency/update policy, integrity and provenance records for executable additions. Add lockfiles and SBOMs when actual packaging/dependencies warrant them.

Reuse: privacy-review, agent-configuration-review, release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Actual executable dependencies and their licences; do not invent an SBOM need for prose alone.

Observable acceptance: Lockfiles or SBOMs describe real packaged dependencies; no speculative supply-chain infrastructure for nonexistent services.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 076, Supply-chain records. Reuse the existing coordinator routes and relevant available specialist/provider first. Dependency/update policy, integrity and provenance records for executable additions. Add lockfiles and SBOMs when actual packaging/dependencies warrant them. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse privacy-review, agent-configuration-review, release-readiness-and-observability and the references named in this entry. Prerequisites: Actual executable dependencies and their licences; do not invent an SBOM need for prose alone. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

077. Licensing and redistribution decision

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Audit every adaptation and dependency; retain notices, document provenance and identify material that cannot be redistributed. Choose licensing for original work only after compatibility review.

Reuse: privacy-review, agent-configuration-review, release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Produce a concrete distribution decision per affected source; never infer permission from public availability or apply one license blindly.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 077, Licensing and redistribution decision. Reuse the existing coordinator routes and relevant available specialist/provider first. Audit every adaptation and dependency; retain notices, document provenance and identify material that cannot be redistributed. Choose licensing for original work only after compatibility review. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse privacy-review, agent-configuration-review, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

078. Safe private-profile onboarding

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Public synthetic template, minimal setup, field meanings, profile precedence, no-profile fallback and correction/deletion/backup instructions. Never ship personal facts, credentials or private source documents.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Public examples contain no personal data; private profile locations stay outside Git, exports and QMD skill collections.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 078, Safe private-profile onboarding. Reuse the existing coordinator routes and relevant available specialist/provider first. Public synthetic template, minimal setup, field meanings, profile precedence, no-profile fallback and correction/deletion/backup instructions. Never ship personal facts, credentials or private source documents. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

079. Installation doctor

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Read-only checks for source/cache drift, missing routes, metadata, optional tool availability and index freshness. Show a repair preview; do not silently rewrite configuration.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: It reports unknown and blocked states honestly, redacts private paths where appropriate and previews repairs without applying them.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 079, Installation doctor. Reuse the existing coordinator routes and relevant available specialist/provider first. Read-only checks for source/cache drift, missing routes, metadata, optional tool availability and index freshness. Show a repair preview; do not silently rewrite configuration. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

080. Versioning and releases

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Version policy, changelog, release notes, known issues, compatibility statements, migration notes and rollback target. Keep public releases separate from local cache refreshes.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: A maintainer can prepare a release without accidentally publishing, tagging or pushing; known limitations remain visible.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 080, Versioning and releases. Reuse the existing coordinator routes and relevant available specialist/provider first. Version policy, changelog, release notes, known issues, compatibility statements, migration notes and rollback target. Keep public releases separate from local cache refreshes. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

081. Installation lifecycle tests

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Fresh install, upgrade, rollback, stale cache, uninstall, missing optional tools and profile isolation. Start with the actual supported environment; never claim broader compatibility without evidence.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: 079, 075, 080 and a supported isolated install target; private-profile isolation and rollback authority.

Observable acceptance: Record environment, version and discovery evidence. Restore prior state; never delete the real private profile or claim untested OS support.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 081, Installation lifecycle tests. Reuse qa-and-test-evidence and its evaluation reference first. Fresh install, upgrade, rollback, stale cache, uninstall, missing optional tools and profile isolation. Start with the actual supported environment; never claim broader compatibility without evidence. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: 079, 075, 080 and a supported isolated install target; private-profile isolation and rollback authority. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

082. Scoped CI

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Explicitly enabled repository checks with minimal permissions, pinned dependencies/actions and readable failure evidence. Separate cheap structural checks from costly model evaluations; no live accounts or private data.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Structural repository checks, 081 stable checks, authorized repository CI and least-privilege credentials; model runs remain separate.

Observable acceptance: No secrets or paid live accounts are needed. Local configuration is ready for review; remote activation remains a separate action.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 082, Scoped CI. Reuse qa-and-test-evidence and its evaluation reference first. Explicitly enabled repository checks with minimal permissions, pinned dependencies/actions and readable failure evidence. Separate cheap structural checks from costly model evaluations; no live accounts or private data. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: Structural repository checks, 081 stable checks, authorized repository CI and least-privilege credentials; model runs remain separate. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

083. Additional client support

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Claude Code, other harnesses and OS-specific adapters only when there is an actual user need. Document differing tool, hook and permission support.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Support claims are bounded to observed behavior; Codex and Claude hook parity is never assumed.

Verification stage: Supported isolated lifecycle checks when explicitly authorized; no accounts, installation or distribution from document scope alone.

Ordering reason: Finalize supported packaging only after capability contracts stabilize.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 083, Additional client support. Reuse the existing coordinator routes and relevant available specialist/provider first. Claude Code, other harnesses and OS-specific adapters only when there is an actual user need. Document differing tool, hook and permission support. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

E - Stable documentation and release decision

084. Installation guide

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Exact supported Codex installation steps, prerequisites, source registration, fresh-session discovery, expected result and failure recovery. Separate end-user installation from developer setup.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: A reader can follow the guide without private machine paths; all unexecuted installation steps are explicitly labelled.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 084, Installation guide. Reuse documentation-and-adrs and the current README/manifest first. Exact supported Codex installation steps, prerequisites, source registration, fresh-session discovery, expected result and failure recovery. Separate end-user installation from developer setup. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

085. Update, uninstall and rollback guide

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Document the supported update flow, source/cache distinction, version readback, rollback and uninstall. Preserve private profiles and distinguish optional removal of private data.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: The rollback target and recovery sequence are clear; no instructions require editing installed caches directly.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 085, Update, uninstall and rollback guide. Reuse documentation-and-adrs and the current README/manifest first. Document the supported update flow, source/cache distinction, version readback, rollback and uninstall. Preserve private profiles and distinguish optional removal of private data. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

086. README and first-use quickstart

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Clear purpose, audience, support status, install link, first successful task, screenshots only after verification, limitations and navigation. Preserve the useful existing skill inventory.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Every claim matches the package; no placeholder badges, unverified screenshots or invented provider access.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 086, README and first-use quickstart. Reuse documentation-and-adrs and the current README/manifest first. Clear purpose, audience, support status, install link, first successful task, screenshots only after verification, limitations and navigation. Preserve the useful existing skill inventory. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

087. Everyday usage guide

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Coordinator versus direct specialist invocation, supported examples, project context, provider dependencies, approval reuse and how to request tests. Explain what installation does and does not provide.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Examples distinguish mandatory, optional and unnecessary specialist use and work without exposing private facts.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 087, Everyday usage guide. Reuse documentation-and-adrs and the current README/manifest first. Coordinator versus direct specialist invocation, supported examples, project context, provider dependencies, approval reuse and how to request tests. Explain what installation does and does not provide. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

088. Troubleshooting and support guide

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Skill not found, wrong route, duplicate installs, stale updates, missing dependencies, authentication failures, index issues and permission denials. Include safe diagnostic information to share and redact.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Each recovery step is scoped and reversible; the guide identifies safe redacted evidence and never recommends bypassing access restrictions.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 088, Troubleshooting and support guide. Reuse documentation-and-adrs and the current README/manifest first. Skill not found, wrong route, duplicate installs, stale updates, missing dependencies, authentication failures, index issues and permission denials. Include safe diagnostic information to share and redact. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

089. Git/GitHub user recipes

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Worked examples for branches, focused commits, PRs, review, conflicts, divergence, rejected pushes, detached HEAD and interrupted operations. Build on the existing Git skill.

Reuse: git-and-github-workflow. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Every destructive or remote action is distinguished from read-only inspection, with recovery and project-intent ambiguity explained.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 089, Git/GitHub user recipes. Reuse the existing coordinator routes and relevant available specialist/provider first. Worked examples for branches, focused commits, PRs, review, conflicts, divergence, rejected pushes, detached HEAD and interrupted operations. Build on the existing Git skill. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse git-and-github-workflow and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

090. Workflow cookbook

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Copyable coding, research, university, writing, LinkedIn and Git examples with prerequisites, expected outputs and stop conditions. Use synthetic facts and approved public examples.

Reuse: documentation-and-adrs. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Recipes route through existing providers, state stop conditions and show both specialist and no-specialist examples.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 090, Workflow cookbook. Reuse documentation-and-adrs and the current README/manifest first. Copyable coding, research, university, writing, LinkedIn and Git examples with prerequisites, expected outputs and stop conditions. Use synthetic facts and approved public examples. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse documentation-and-adrs and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

091. Examples and demos

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Small synthetic sample projects, before/after walkthroughs and representative outputs. Label illustrative outputs separately from recorded runs.

Reuse: documentation-and-adrs. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Illustrative output is labelled; recorded run claims include actual evidence and contain no private project details.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 091, Examples and demos. Reuse documentation-and-adrs and the current README/manifest first. Small synthetic sample projects, before/after walkthroughs and representative outputs. Label illustrative outputs separately from recorded runs. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse documentation-and-adrs and the references named in this entry.

Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

092. Contributor and maintainer guides

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: How to add/update a skill, route it, preserve licensing, maintain inventories, refresh scoped QMD and update the pilot. Include review and authorized testing procedures.

Reuse: demir-bot, documentation-and-adrs, release-readiness-and-observability. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: One focused contributor example uses existing creators and preserves unrelated changes; no hidden automatic test or commit step.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 092, Contributor and maintainer guides. Reuse documentation-and-adrs and the current README/manifest first. How to add/update a skill, route it, preserve licensing, maintain inventories, refresh scoped QMD and update the pilot. Include review and authorized testing procedures. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse demir-bot, documentation-and-adrs, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

093. Security reporting policy

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Supported scope, vulnerability-reporting route, safe disclosure and secret-exposure recovery. Specify a real private contact before publishing a reporting policy.

Reuse: privacy-review, agent-configuration-review, release-readiness-and-observability. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: A real destination is supplied before publication; no invented contact, response SLA or guarantee of vulnerability absence.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 093, Security reporting policy. Reuse the existing coordinator routes and relevant available specialist/provider first. Supported scope, vulnerability-reporting route, safe disclosure and secret-exposure recovery. Specify a real private contact before publishing a reporting policy. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in `plugins/demir-bot-pilot`, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse privacy-review, agent-configuration-review, release-readiness-and-observability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

094. GitHub collaboration setup

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Issue forms, PR template, useful labels and ownership rules when collaborators exist. Evaluate branch protections and required checks separately; current remote settings were not audited.

Reuse: git-and-github-workflow. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Templates discourage posting secrets; remote labels, protections and ownership changes are proposals until separately authorized.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 094, GitHub collaboration setup. Reuse the existing coordinator routes and relevant available specialist/provider first. Issue forms, PR template, useful labels and ownership rules when collaborators exist. Evaluate branch protections and required checks separately; current remote settings were not audited. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse git-and-github-workflow and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

095. Multilingual documentation

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Translations and localized examples after the primary installation/usage docs stabilize. Preserve technical meaning and maintenance ownership.

Reuse: documentation-and-adrs. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: 084, 086, 087, 085, 078, 088 stable primary-language docs and approved translations.

Observable acceptance: Terminology, commands and approval boundaries remain accurate and ownership for future updates is clear.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 095, Multilingual documentation. Reuse documentation-and-adrs and the current README/manifest first. Translations and localized examples after the primary installation/usage docs stabilize. Preserve technical meaning and maintenance ownership. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse documentation-and-adrs and the references named in this entry. Prerequisites: 084, 086, 087, 085, 078, 088 stable primary-language docs and approved translations. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

096. Documentation site and richer presentation

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Searchable guide site, diagrams, demo video, branding and verified badges after core docs work. Static repository docs are sufficient initially.

Reuse: documentation-and-adrs. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: 084, 086, 087, 085, 078, 088 and 090 stable documentation; hosting/publication remains separately authorized.

Observable acceptance: Content remains source-linked and accessible; hosting, branding assets and publication are separately scoped.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 096, Documentation site and richer presentation. Reuse documentation-and-adrs and the current README/manifest first. Searchable guide site, diagrams, demo video, branding and verified badges after core docs work. Static repository docs are sufficient initially. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse documentation-and-adrs and the references named in this entry. Prerequisites: 084, 086, 087, 085, 078, 088 and 090 stable documentation; hosting/publication remains separately authorized. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

097. Public-project community support

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Discussions, issue taxonomy, contributor recognition, governance, code of conduct and funding links if public participation becomes a goal.

Reuse: documentation-and-adrs. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: A real public participation goal and 092/093 governance/contact decisions.

Observable acceptance: Contacts, roles and funding destinations are real; no fabricated response commitments or automatic public settings changes.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 097, Public-project community support. Reuse the existing coordinator routes and relevant available specialist/provider first. Discussions, issue taxonomy, contributor recognition, governance, code of conduct and funding links if public participation becomes a goal. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse documentation-and-adrs and the references named in this entry. Prerequisites: A real public participation goal and 092/093 governance/contact decisions. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

098. Release acceptance and recovery

Status: REMAINING; existing guidance may satisfy part of the scope.

Purpose and remaining deliverable: Concrete criteria, known limitations, provider failure handling, rollback and recovery evidence. Reuse release-readiness guidance; execute checks only within granted scope.

Reuse: release-readiness-and-observability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: C validation, D lifecycle evidence and E stable docs; confidentiality, licensing and unresolved release risks.

Observable acceptance: Source review, execution evidence and human release authorization are separate; no unsupported production-ready claim.

Verification stage: Document/source consistency and authorized rendering; verify operational examples only under separate scope. Final release evidence at 101.

Ordering reason: Describe stable behavior; public polish and release approval follow evidence.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to implement roadmap item 098, Release acceptance and recovery. Reuse the existing coordinator routes and relevant available specialist/provider first. Concrete criteria, known limitations, provider failure handling, rollback and recovery evidence. Reuse release-readiness guidance; execute checks only within granted scope. Use the scope in this roadmap entry; inspect current coverage and extend an existing skill/reference when sufficient, creating a new namespaced skill only for a distinct uncovered capability. Work in plugins/demir-bot-pilot, preserve unrelated work and private data, and keep the coordinator minimal. Do not run tests, builds, evaluations or benchmarks, commit/push, install external providers, publish, spend or change accounts. Writing checks is authorized only if the requested deliverable is a check suite. If a project or access dependency is missing, state it rather than invent it. Report saved changes and unverified behavior; do not refresh the installed cache or resume cloud sync, RAG, training or background automation. Reuse release-readiness-and-observability and the references named in this entry. Prerequisites: C validation, D lifecycle evidence and E stable docs; confidentiality, licensing and unresolved release risks. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

F - Explicit deferrals and reuse decisions

099. Duplicate debugging specialist

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Superpowers systematic debugging is already available. Do not create a parallel debugging skill just for catalogue parity. Verify selection, scope, evidence handling and fallback through priority-1 Precise routing, negative cases and composition instead.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Only a demonstrated routing defect warrants a focused change under Precise routing, negative cases and composition; keeping the current provider is a valid outcome.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 099, Duplicate debugging specialist. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

100. Duplicate skill-authoring toolkit

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Existing skill-creator and plugin-creator capabilities cover authoring. Reuse them; maintain only Demir Bot-specific routing, provenance and update instructions in the contributor guide. Avoid creating another competing authoring workflow.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Only project-specific documentation or a demonstrated routing gap is changed; duplicate tooling is not created.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 100, Duplicate skill-authoring toolkit. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

101. Autonomous multi-agent loops

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Dev fleets, councils, GAN-style critique, orchestration pipelines, recursive ledgers and agent teams add complexity and spend. Use bounded delegation only when requested and useful.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Any reconsideration names a concrete task, budget, stop condition and evidence that delegation helps.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Coordination decision: permanent nested coordinators remain deferred. Reconsider only after repeated evidence of a coordination bottleneck and an authorized comparison showing that simpler routing or one-layer delegation is insufficient. Routine domain routing is not an autonomous multi-agent loop.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 101, Autonomous multi-agent loops. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior. Coordination decision: permanent nested coordinators remain deferred. Reconsider only after repeated evidence of a coordination bottleneck and an authorized comparison showing that simpler routing or one-layer delegation is insufficient. Routine domain routing is not an autonomous multi-agent loop.

102. Background learning and persona mutation

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Continuous observation, instincts, automatic rule distillation and self-modification conflict with explicit persistence and privacy boundaries. Keep on-demand approved improvements.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Any future change requires explicit privacy, retention and persistence scope; silence never becomes permission.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 102, Background learning and persona mutation. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

103. Automatic maintenance and monitoring

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Auto-updates, canary watchers, recurring audits, scheduled pruning and automatic notification loops remain inactive without an explicit request.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: A future automation must specify schedule/event, notification policy, owner and stop condition before activation.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 103, Automatic maintenance and monitoring. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

104. Mandatory testing/verification loops

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Do not import blanket TDD, mandatory browser/build checks, arbitrary coverage thresholds or endless fix/retest loops. Build testing capability while preserving opt-in use.

Reuse: qa-and-test-evidence. Canonical shared references: plugins/demir-bot-pilot/skills/demir-bot/references/routes.md; plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Testing follows explicit task authorization and meaningful risk; no upstream workflow overrides user scope.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 104, Mandatory testing/verification loops. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse qa-and-test-evidence and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

105. Cloud sync and university RAG

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Existing explicit deferrals remain. University retrieval depends on the intended dashboard/source access and a separately approved design.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Explicit resumed scope, intended university dashboard/source access and approved retrieval/privacy design.

Observable acceptance: Resumption requires explicit scope and available dashboard/source access; an unrelated connector request does not resume both.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 105, Cloud sync and university RAG. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Explicit resumed scope, intended university dashboard/source access and approved retrieval/privacy design. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

106. Training and compute infrastructure

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Remote training, inference/compute marketplaces and recommendation infrastructure are not required by the plugin itself. Revisit for a concrete authorized ML project.

Reuse: ml-training-specialist, swiftui-performance-and-concurrency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Explicit resumed training scope, a concrete ML project, authorized compute and data rights.

Observable acceptance: A future ML project requires a trainable model, permitted data, evaluation design, compute budget and explicit execution authority.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 106, Training and compute infrastructure. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse ml-training-specialist, swiftui-performance-and-concurrency and the references named in this entry. Prerequisites: Explicit resumed training scope, a concrete ML project, authorized compute and data rights. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

107. Healthcare and regulated clinical packs

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Clinical decision support, EMR, PHI/HIPAA and healthcare evaluations need domain-specific expertise and a real project.

Reuse: deep-research-and-idea-validation, jurisdiction-specific-legal-feasibility. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Reassessment identifies jurisdiction, data sensitivity, expertise and validation needs without implying clinical readiness.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 107, Healthcare and regulated clinical packs. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse deep-research-and-idea-validation, jurisdiction-specific-legal-feasibility and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

108. Crypto, trading and agent payments

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: DeFi, token arithmetic, prediction markets, trading agents and payment protocols are unrelated to the current product scope.

Reuse: deep-research-and-idea-validation, jurisdiction-specific-legal-feasibility. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Reassessment separates research from financial action, credentials, payments and economic authorization.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 108, Crypto, trading and agent payments. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse deep-research-and-idea-validation, jurisdiction-specific-legal-feasibility and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

109. Logistics, manufacturing and enterprise operations

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Carriers, customs, procurement, demand planning, production scheduling, returns, billing and quality systems need actual business requirements.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: A real workflow, data owner and operational boundary must justify any later addition.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 109, Logistics, manufacturing and enterprise operations. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

110. Homelab and network administration

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Cisco, BGP, VLAN, VPN, DNS and SSH fleet automation are optional domain tooling, not missing personal-assistant foundations.

Reuse: architecture-review, requirements-and-traceability. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Future work names devices, ownership, access, backup and recovery before configuration changes.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 110, Homelab and network administration. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse architecture-review, requirements-and-traceability and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

111. Duplicate provider implementations

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: Do not rebuild document, PDF, search, design, media, security or native task tools already available. Improve routes and usage guides instead.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references:

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md;

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Use a coverage comparison and demonstrated gap; duplication for feature count is rejected.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 111, Duplicate provider implementations. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

112. ECC runtime and command parity

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: No need to copy every command alias, control plane, agent persona or external-runtime pointer. Adopt unique useful behavior, not catalogue size.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts.

Observable acceptance: Adopt only unique useful behavior with compatible permissions, dependencies, licensing and measurable value.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 112, ECC runtime and command parity. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Current source and the named owner instructions; inspect existing coverage first. Preserve the governing permission and private-data contracts. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

113. Early-window scheduling idea - reassess only

Status: DEFERRED / REUSE-ONLY; reassessment is not activation.

Purpose and remaining deliverable: The video suggests a small early-morning scheduled request to start a usage window. This is not a core skill or an established savings result. Keep disabled unless explicitly requested after checking current plan/reset behavior. Explain whether the scheduling suggestion is useful for the actual account; do not create automation from this roadmap.

Reuse: demir-bot, agent-configuration-review, context-efficiency. Canonical shared references: `plugins/demir-bot-pilot/skills/demir-bot/references/routes.md`; `plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md`. Use each owner's SKILL.md and its task-specific references; new namespaced skills require a distinct uncovered trigger.

Prerequisites and blockers: Explicit scheduling request and current plan/reset evidence; no proven savings.

Observable acceptance: Current mechanics, usage cost and user-authorized schedule are known before any setup.

Verification stage: Read-only source/proposal review; any activation requires a new explicit request.

Ordering reason: No demonstrated current need or explicit deferral; avoid duplicate services.

Ready-to-copy prompt:

Use Demir Bot - Private Pilot to reassess roadmap item 113, Early-window scheduling idea - reassess only. Review the retained scope and acceptance contract in this roadmap. Produce a concise recommendation only; do not activate, install, implement, schedule or run its deferred capability. Reuse demir-bot, agent-configuration-review, context-efficiency and the references named in this entry. Prerequisites: Explicit scheduling request and current plan/reset evidence; no proven savings. Do not write checks unless this item explicitly requests a check suite or fixture; writing never implies running. Do not delegate or run model evaluations without explicit bounded authority. Do not refresh the installed cache or QMD indexes. Keep cloud sync, university RAG, training and background automation inactive. No commit/push, publication, provider installation, spending, account changes or private-data modification. Report saved changes separately from observed operations and unverified behavior.

Repository artifact inventory and maintenance

This inventory covers the plugin package, not private data or unrelated checkout files. README.md describes the repository and RESOURCES.md records provenance. Retain upstream notices; private use is not a blanket redistribution licence. The editable Markdown is the document source; edit it and regenerate the PDF using the accompanying renderer. The previous PDF remains historical evidence. Regeneration does not update installed skills or implement roadmap items.

plugins/demir-bot-pilot/.codex-plugin/plugin.json

plugins/demir-bot-pilot/skills/agent-configuration-review/SKILL.md

plugins/demir-bot-pilot/skills/agent-configuration-review/agents/openai.yaml

plugins/demir-bot-pilot/skills/agent-configuration-review/references/local-scanner.md

plugins/demir-bot-pilot/skills/agent-configuration-review/references/provenance.md

plugins/demir-bot-pilot/skills/architecture-review/LICENSE

plugins/demir-bot-pilot/skills/architecture-review/SKILL.md

plugins/demir-bot-pilot/skills/architecture-review/agents/openai.yaml

plugins/demir-bot-pilot/skills/architecture-review/references/review-checklist.md

plugins/demir-bot-pilot/skills/architecture-review/references/sources.md

plugins/demir-bot-pilot/skills/clarify-and-execute/LICENSE.upstream

plugins/demir-bot-pilot/skills/clarify-and-execute/SKILL.md

plugins/demir-bot-pilot/skills/clarify-and-execute/agents/openai.yaml

plugins/demir-bot-pilot/skills/clarify-and-execute/references/choices.md

plugins/demir-bot-pilot/skills/clarify-and-execute/references/sources.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/LICENSE

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/SKILL.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/agents/openai.yaml

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/contracts-and-documentation.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/course-lessons.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/quality-review.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/sources.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/targeted-review-provenance.md

plugins/demir-bot-pilot/skills/code-refactoring-refactor-clean/references/testing-and-evidence.md

plugins/demir-bot-pilot/skills/context-efficiency/LICENSE.upstream

plugins/demir-bot-pilot/skills/context-efficiency/SKILL.md

plugins/demir-bot-pilot/skills/context-efficiency/agents/openai.yaml

plugins/demir-bot-pilot/skills/context-efficiency/references/runtime-options.md

plugins/demir-bot-pilot/skills/context-efficiency/references/source-reuse.md

plugins/demir-bot-pilot/skills/context-efficiency/references/sources.md

plugins/demir-bot-pilot/skills/deep-research-and-idea-validation/SKILL.md

plugins/demir-bot-pilot/skills/deep-research-and-idea-validation/agents/openai.yaml

plugins/demir-bot-pilot/skills/deep-research-and-idea-validation/references/provenance.md

plugins/demir-bot-pilot/skills/demir-bot/SKILL.md

plugins/demir-bot-pilot/skills/demir-bot/agents/openai.yaml

plugins/demir-bot-pilot/skills/demir-bot/references/capability-maintenance.md

plugins/demir-bot-pilot/skills/demir-bot/references/capability-register.md

plugins/demir-bot-pilot/skills/demir-bot/references/delivery.md

plugins/demir-bot-pilot/skills/demir-bot/references/execution-scope.md

plugins/demir-bot-pilot/skills/demir-bot/references/personal-workflows.md

plugins/demir-bot-pilot/skills/demir-bot/references/plugin-routes.md

plugins/demir-bot-pilot/skills/demir-bot/references/profile-loading.md

plugins/demir-bot-pilot/skills/demir-bot/references/routes.md

plugins/demir-bot-pilot/skills/demir-bot/references/skill-sync.md

plugins/demir-bot-pilot/skills/demir-bot/references/upstream-adaptations.md

plugins/demir-bot-pilot/skills/demir-linkedin/SKILL.md

plugins/demir-bot-pilot/skills/demir-linkedin/agents/openai.yaml

plugins/demir-bot-pilot/skills/demir-linkedin/references/actions.md

plugins/demir-bot-pilot/skills/demir-linkedin/references/approval-ledger.md

plugins/demir-bot-pilot/skills/demir-linkedin/references/growth.md

plugins/demir-bot-pilot/skills/demir-linkedin/references/publishing-runtime.md

plugins/demir-bot-pilot/skills/demir-linkedin/references/research-and-content.md

plugins/demir-bot-pilot/skills/demir-linkedin/references/sources.md

plugins/demir-bot-pilot/skills/documentation-and-adrs/LICENSE

plugins/demir-bot-pilot/skills/documentation-and-adrs/SKILL.md

plugins/demir-bot-pilot/skills/documentation-and-adrs/agents/openai.yaml

plugins/demir-bot-pilot/skills/documentation-and-adrs/references/adr-and-templates.md

plugins/demir-bot-pilot/skills/documentation-and-adrs/references/consistency-and-evidence.md

plugins/demir-bot-pilot/skills/documentation-and-adrs/references/sources.md

plugins/demir-bot-pilot/skills/email-tone/SKILL.md

plugins/demir-bot-pilot/skills/email-tone/agents/openai.yaml

plugins/demir-bot-pilot/skills/frontend-quality-and-accessibility/SKILL.md

plugins/demir-bot-pilot/skills/frontend-quality-and-accessibility/agents/openai.yaml

plugins/demir-bot-pilot/skills/frontend-quality-and-accessibility/references/provenance.md

plugins/demir-bot-pilot/skills/git-and-github-workflow/LICENSE.upstream

plugins/demir-bot-pilot/skills/git-and-github-workflow/SKILL.md

plugins/demir-bot-pilot/skills/git-and-github-workflow/agents/openai.yaml

plugins/demir-bot-pilot/skills/git-and-github-workflow/references/provenance.md

plugins/demir-bot-pilot/skills/git-and-github-workflow/references/recovery.md

plugins/demir-bot-pilot/skills/jurisdiction-specific-legal-feasibility/SKILL.md

plugins/demir-bot-pilot/skills/jurisdiction-specific-legal-feasibility/agents/openai.yaml

plugins/demir-bot-pilot/skills/jurisdiction-specific-legal-feasibility/references/provenance.md

plugins/demir-bot-pilot/skills/linkedin-tone/SKILL.md

plugins/demir-bot-pilot/skills/linkedin-tone/agents/openai.yaml

plugins/demir-bot-pilot/skills/ml-training-specialist/SKILL.md

plugins/demir-bot-pilot/skills/ml-training-specialist/agents/openai.yaml

plugins/demir-bot-pilot/skills/ml-training-specialist/references/provenance.md

plugins/demir-bot-pilot/skills/personal-writing/SKILL.md

plugins/demir-bot-pilot/skills/personal-writing/agents/openai.yaml

plugins/demir-bot-pilot/skills/personal-writing/references/facts-and-voice.md

plugins/demir-bot-pilot/skills/privacy-review/SKILL.md

plugins/demir-bot-pilot/skills/privacy-review/agents/openai.yaml

plugins/demir-bot-pilot/skills/privacy-review/references/sources-and-coverage.md

plugins/demir-bot-pilot/skills/qa-and-test-evidence/LICENSE

plugins/demir-bot-pilot/skills/qa-and-test-evidence/SKILL.md

plugins/demir-bot-pilot/skills/qa-and-test-evidence/agents/openai.yaml

plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/demir-bot-evaluation.md

plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/evidence.md

plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/sources.md

plugins/demir-bot-pilot/skills/qa-and-test-evidence/references/test-design.md

plugins/demir-bot-pilot/skills/release-readiness-and-observability/SKILL.md

plugins/demir-bot-pilot/skills/release-readiness-and-observability/agents/openai.yaml

plugins/demir-bot-pilot/skills/release-readiness-and-observability/references/provenance.md

plugins/demir-bot-pilot/skills/requirements-and-traceability/LICENSE

plugins/demir-bot-pilot/skills/requirements-and-traceability/SKILL.md

plugins/demir-bot-pilot/skills/requirements-and-traceability/agents/openai.yaml

plugins/demir-bot-pilot/skills/requirements-and-traceability/references/requirements.md

plugins/demir-bot-pilot/skills/requirements-and-traceability/references/sources.md

plugins/demir-bot-pilot/skills/requirements-and-traceability/references/traceability.md

plugins/demir-bot-pilot/skills/swiftui-performance-and-concurrency/SKILL.md

plugins/demir-bot-pilot/skills/swiftui-performance-and-concurrency/agents/openai.yaml

plugins/demir-bot-pilot/skills/swiftui-performance-and-concurrency/references/provenance.md

plugins/demir-bot-pilot/skills/youtube-learning/SKILL.md

plugins/demir-bot-pilot/skills/youtube-learning/agents/openai.yaml

[plugins/demir-bot-pilot/skills/youtube-learning/references/provenance.md](#)

[plugins/demir-bot-pilot/skills/youtube-learning/references/providers.md](#)

Evidence gaps and next eligible work

Next remaining entry: 001, Swift concurrency, state ownership and identity. It is conditional on a named Swift project and a demonstrated gap. Inspect existing coverage first and reuse sufficient guidance. Without that project dependency, select another applicable remaining item rather than inventing project work. Completed capabilities are retained above as current state.

Remaining evidence gaps include durable original assessment artifacts, full reference/domain behavioral coverage, fresh-task discovery, live provider boundaries, complete lifecycle recovery, public licensing compatibility and comparable cost baselines. The historical PDF preserves its external source notes; external video/ECC figures were not re-researched for this local document task and are not asserted as current facts. The future roadmap contains 113 sequentially numbered entries, including explicitly deferred and reuse-only decisions. No new plugin reliability, measured savings or release-readiness claim is made.